UNIVERSITÀ DI FIRENZE

DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

Corso di Laurea Triennale in

Ingegneria Informatica

BOOKTRACK

Software per la gestione e condivisione di libri e recensioni

Professore

Vicario Enrico

Studente

KONDJI NGUIMBOUS JUNIOR

Indice

1 Introduzione 3
1.1 Obiettivo e descrizione del progetto 3
1.2 Struttura e pratiche utilizzate 3

2 Progettazione 4
2.1 Use Case Diagram 4
2.2 Use Case Template 6
2.3 Mockups 9
2.4 Class Diagram 15
2.5 ER Diagram e modello relazionale 19
2.6 Navigation Diagram 21
2.7 Struttura delle directory 24

3 Implementazione 25
3.1 CoreEntities 25
3.1.1 User 25
3.1.2 Book 26
3.1.3 Request 27
3.1.4 Reservation e ReservationMapper 29
3.1.5 PaymentStrategy, CreditCard e PayPal 29
3.2 LibraryManagement 29
3.2.1 LoginController 29
3.2.2 UserBookPageController 29
3.2.3 UserBookManagementController 29
3.2.4 UserProfileController 30
3.2.5 AdminBookController 30
3.2.6 AdminUserController 30
3.2.7 AdminExtraController 31
3.3 Object-Relational Mapping 32
3.3.1 DatabaseConnectionManager 33
3.3.2 UserDAO e AdminDAO 34
3.3.3 BookDAO 35
3.3.4 ReservationDAO e BorrowingDAO 38
3.3.5 CreditCardDAO e PayPalDAO 38
3.4 Database 40
3.5 Interfaccia CLI 41

4 Testing 43
4.1 LibraryManagement Test 43
4.2 CoreEntities Test 48

1 Introduzione

Elaborato per il superamento dell'esame di Ingegneria del Software, appartenente al modulo *Basi di Dati / Ingegneria del Software* del corso di Laurea Triennale in Ingegneria Informatica dell'Università degli Studi di Firenze.

Il progetto è stato sviluppato da **Kondji Nguimbous Junior**, matricola 7090449, durante il periodo di **Gennaio - Giugno 2025**.

Il codice sorgente è disponibile su GitHub al seguente indirizzo: https://github.com/JUNIOR ingegneria-del-software-kondji[1]

1.1 Obiettivo e descrizione del progetto

Il progetto mira a sviluppare una piattaforma che consenta agli utenti di creare una libreria personale di libri, condividere recensioni e scoprire nuovi libri attraverso le raccomandazioni della community.

Gli utenti possono aggiungere libri, scrivere recensioni, seguire altri utenti e gestire il proprio profilo.

1.2 Struttura e pratiche utilizzate

Il software è stato sviluppato in **Java**, con la gestione dei dati implementata tramite un database **PostgreSQL** utilizzando la libreria **JDBC (Java DataBase Connectivity)**.

Per garantire la separazione delle responsabilità, il progetto è stato organizzato in tre pacchetti principali:

- `LibraryManagement`: implementa la logica di business del sistema.
- `CoreEntities`: rappresenta le entità principali del sistema.
- `ORM (Object-Relational Mapping)`: si occupa dell'accesso ai dati e della loro persistenza.

Gli strumenti e le piattaforme utilizzati includono:

- **IntelliJ IDEA**: IDE per lo sviluppo in Java.
- **StarUML** e **Draw.io**: per la creazione di diagrammi UML.
- **PgAdmin**: gestione del database PostgreSQL.

[1] `https://github.com/JUNIORKONDJI/elaborato-ingegneria-del-software-kondji`

- **GitHub**: per il versionamento e la condivisione del codice sorgente.
- **JUnit**: per il testing del sistema.

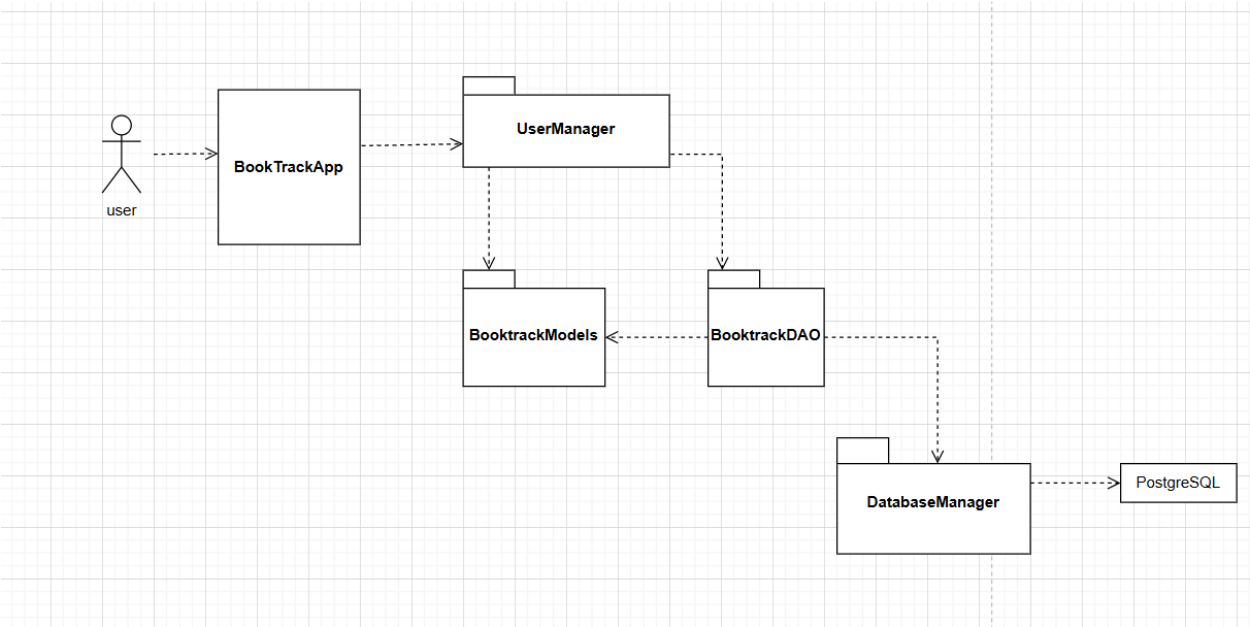


Figura 1: Package Dependency Diagram

2 Progettazione

2.1 Use Case Diagram

Come precedentemente descritto, il sistema è stato progettato per permettere agli utenti di creare librerie personali, condividere recensioni e gestire il proprio profilo. La Figura 2 mostra il diagramma dei casi d'uso principali del sistema, comprendendo sia quelli degli utenti che quelli degli amministratori.

Il sistema prevede i seguenti casi d'uso principali:

- Registrazione e login dell'utente.
- Creazione e gestione della libreria personale.
- Aggiunta e modifica di recensioni per i libri.
- Interazione con altri utenti (seguire e commentare).
- Ricerca e visualizzazione di libri e recensioni.

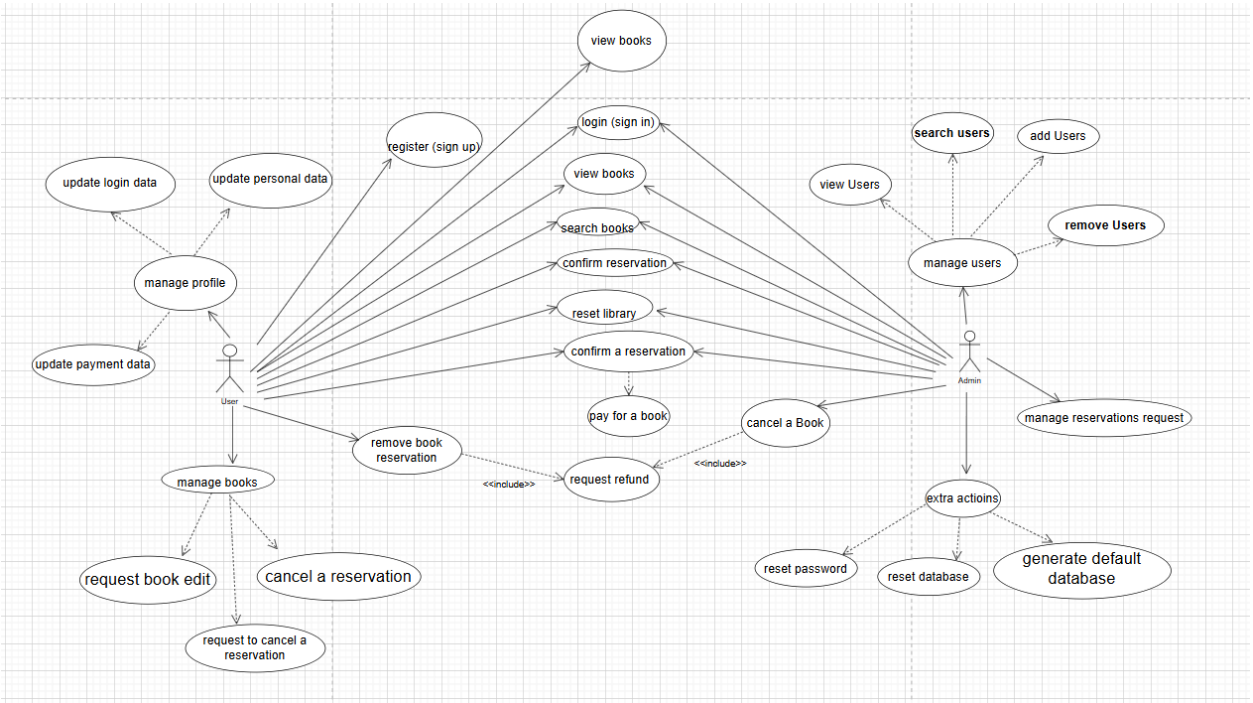


Figura 2: Use Case Diagram

2.2 Use Case Template

In questa sezione vengono descritti nel dettaglio i principali casi d'uso del sistema **BookTrack**, tramite la tecnica degli *Use Case Template*. Ogni caso d'uso

rappresenta una specifica interazione tra uno o più attori (Utente, Admin) e il sistema, con l'obiettivo di portare a termine una funzionalità.

Di seguito è riportato un esempio relativo al caso d'uso di accesso al sistema.

Use Case #1: Accesso al sistema (Sign in)	
Brief Description:	L'utente accede al sistema BookTrack tramite le proprie credenziali (MK#1, TST#01).
Level:	User Goal
Actors:	Utente, Admin
Pre-conditions:	L'utente deve essere nella pagina iniziale di accesso al sistema.
Basic Flow:	1. L'utente inserisce le proprie credenziali (username e password). 2. L'utente preme il pulsante di accesso. 3. Il sistema verifica le credenziali fornite. 4. **Il sistema autentica l'utente.**
Alternative Flows:	• **3a.** Se le credenziali fornite non sono corrette, il sistema mostra un messaggio di errore all'utente. • **4a.** Se a fare l'accesso è l'amministratore (Admin), il sistema lo reindirizza alla pagina di amministrazione *(Admin Page)* (TST#03).
Post-conditions:	L'utente è autenticato nel sistema e ha accesso alle funzionalità riservate.

Tabella 1: Use Case #1 – Accesso al sistema (Sign in)

Use Case #2: Registrazione del sistema (Sign up)	
Brief Description:	L'utente si registra nel sistema BookTrack creando un nuovo account per accedere alle funzionalità della piattaforma (MK#2, TST#02).
Level:	User Goal
Actors:	Utente
Pre-conditions:	L'utente deve essere nella pagina iniziale di accesso al sistema.
Basic Flow:	1. L'utente fornisce i dettagli richiesti per la registrazione (nome utente, email, password). 2. L'utente conferma la registrazione premendo il pulsante "Registrati". 3. Il sistema verifica i dati forniti e controlla che l'email e il nome utente non siano già utilizzati. 4. Il sistema crea un nuovo account per l'utente.
Alternative Flows:	• Se l'utente fornisce dati non validi o se l'email o il nome utente sono già utilizzati da un altro account, il sistema mostra un messaggio di errore e richiede la correzione dei dati.
Post-conditions:	L'utente è registrato nel sistema e può accedere utilizzando le credenziali create durante la registrazione.

Tabella 2: Use Case #2 – Registrazione del sistema (Sign up)

Use Case #3: Partecipazione a un libro condiviso	
Brief Description:	L'utente partecipa alla discussione di un libro condiviso nella piattaforma (MK#3, TST#04).
Level:	User Goal
Actors:	Utente
Pre-conditions:	L'utente deve trovarsi nella pagina di un libro condiviso e inviare una richiesta per partecipare alla discussione del libro.
Basic Flow:	1. L'utente seleziona un libro condiviso tra quelli disponibili. 2. L'utente conferma la partecipazione alla discussione. 3. Il sistema registra la partecipazione e apre la schermata della discussione per quel libro.
Alternative Flows:	• **2a.** Se l'utente è già registrato alla discussione del libro, il sistema mostra un messaggio informativo. • **3a.** Se il libro non richiede approvazione per partecipare, il sistema registra automaticamente l'utente alla discussione e lo reindirizza alla schermata.
Post-conditions:	L'utente conferma la partecipazione alla discussione.

Tabella 3: Use Case #3 – Partecipazione a un libro condiviso

Use Case #4: Pagamento della quota di prenotazione	
Brief Description:	L'utente effettua il pagamento per la prenotazione di un libro (MK#4, TST#13).
Level:	Function
Actors:	Utente
Pre-conditions:	L'utente ha confermato la prenotazione di un libro che richiede il pagamento.
Basic Flow:	1. Il sistema mostra il riepilogo del pagamento (titolo libro, importo, metodo). 2. L'utente inserisce i dati della carta di credito oppure l'indirizzo e la password PayPal. 3. Il sistema verifica i dati e processa il pagamento. 4. Il sistema conferma all'utente che il pagamento è stato completato con successo.
Alternative Flows:	• **3a.** Se i dati inseriti sono errati, il sistema mostra un messaggio di errore.
Post-conditions:	Il pagamento è stato registrato e la prenotazione del libro è confermata.

Tabella 4: Use Case #4 – Pagamento della quota di prenotazione

Use Case #5: Inserimento di un nuovo libro	
Brief Description:	L'utente o l'amministratore inserisce un nuovo libro nella libreria del sistema (MK#5, TST#06).
Level:	User Goal
Actors:	User, Admin
Pre-conditions:	Il sistema deve trovarsi nella pagina di gestione della libreria (Library Management Page).
Basic Flow:	1. L'utente seleziona l'opzione per aggiungere un nuovo libro. 2. L'utente inserisce i dettagli del libro (titolo, autore, anno, genere, descrizione, disponibilità). 3. L'utente conferma l'inserimento. 4. Il sistema salva il libro nel database.
Alternative Flows:	• **2a.** Se l'utente inserisce dati non validi, il sistema mostra un messaggio di errore e richiede correzione. • **2b.** Se il titolo del libro è già presente, il sistema segnala il conflitto e impedisce il salvataggio.
Post-conditions:	Il libro è stato correttamente aggiunto al sistema e risulta disponibile per la prenotazione.

Tabella 5: Use Case #5 – Inserimento di un nuovo libro

Use Case #6: Cancellazione di una prenotazione	
Brief Description:	L'utente cancella una prenotazione di un libro effettuata in precedenza (TST#05).
Level:	User Goal
Actors:	User
Pre-conditions:	L'utente deve avere già prenotato un libro.
Basic Flow:	1. L'utente seleziona l'opzione per cancellare una prenotazione. 2. L'utente fornisce il codice della prenotazione. 3. Il sistema annulla la prenotazione. 4. Il sistema conferma la cancellazione e, se previsto, rimborsa l'utente.
Alternative Flows:	• **2a.** Se il codice fornito non è valido, il sistema mostra un messaggio di errore. • **4a.** Se la prenotazione non è rimborsabile, viene comunque annullata ma senza rimborso.
Post-conditions:	L'utente non ha più la prenotazione attiva nel sistema.

Tabella 6: Use Case #6 – Cancellazione di una prenotazione

Use Case #7: Aggiornamento del metodo di pagamento	
Brief Description:	L'utente modifica il metodo di pagamento associato al proprio account (TST#08, TST#09).
Level:	User Goal
Actors:	User
Pre-conditions:	L'utente deve essere autenticato nel sistema.
Basic Flow:	1. L'utente accede alle impostazioni del proprio profilo. 2. Seleziona l'opzione per aggiornare il metodo di pagamento. 3. Inserisce i dati del nuovo metodo (es. carta o PayPal). 4. Conferma l'aggiornamento. 5. Il sistema aggiorna le informazioni nel database.
Alternative Flows:	• **3a.** Se i dati inseriti non sono validi, il sistema mostra un messaggio di errore.
Post-conditions:	Il nuovo metodo di pagamento è stato salvato e associato all'utente.

Tabella 7: Use Case #7 – Aggiornamento del metodo di pagamento

Use Case #8: Reset database	
Brief Description:	L'amministratore esegue il ripristino del database del sistema (TST#12).
Level:	Function
Actors:	Admin
Pre-conditions:	L'amministratore ha effettuato l'accesso al sistema.
Basic Flow:	1. L'amministratore seleziona l'opzione per ripristinare il database. 2. Conferma l'operazione. 3. Il sistema esegue il ripristino. 4. Il sistema conferma il completamento dell'operazione.
Alternative Flows:	• **3a.** In caso di errore nel processo di ripristino, il sistema notifica l'amministratore.
Post-conditions:	Il database è stato ripristinato con successo.

Tabella 8: Use Case #8 – Reset del database

2.3 Mockups

Ecco di seguito alcuni possibili *mock-up* relativi alle interfacce grafiche del sistema. I mock-up sono stati realizzati per illustrare l'aspetto e la disposizione delle principali funzionalità accessibili da parte dell'utente e dell'amministratore.

2.4 Mockups

Ecco di seguito alcuni possibili *mock-up* relativi alle interfacce grafiche del sistema. Essi rappresentano le principali schermate a disposizione dell'utente e dell'amministratore per le funzionalità di accesso, prenotazione, pagamento e gestione.

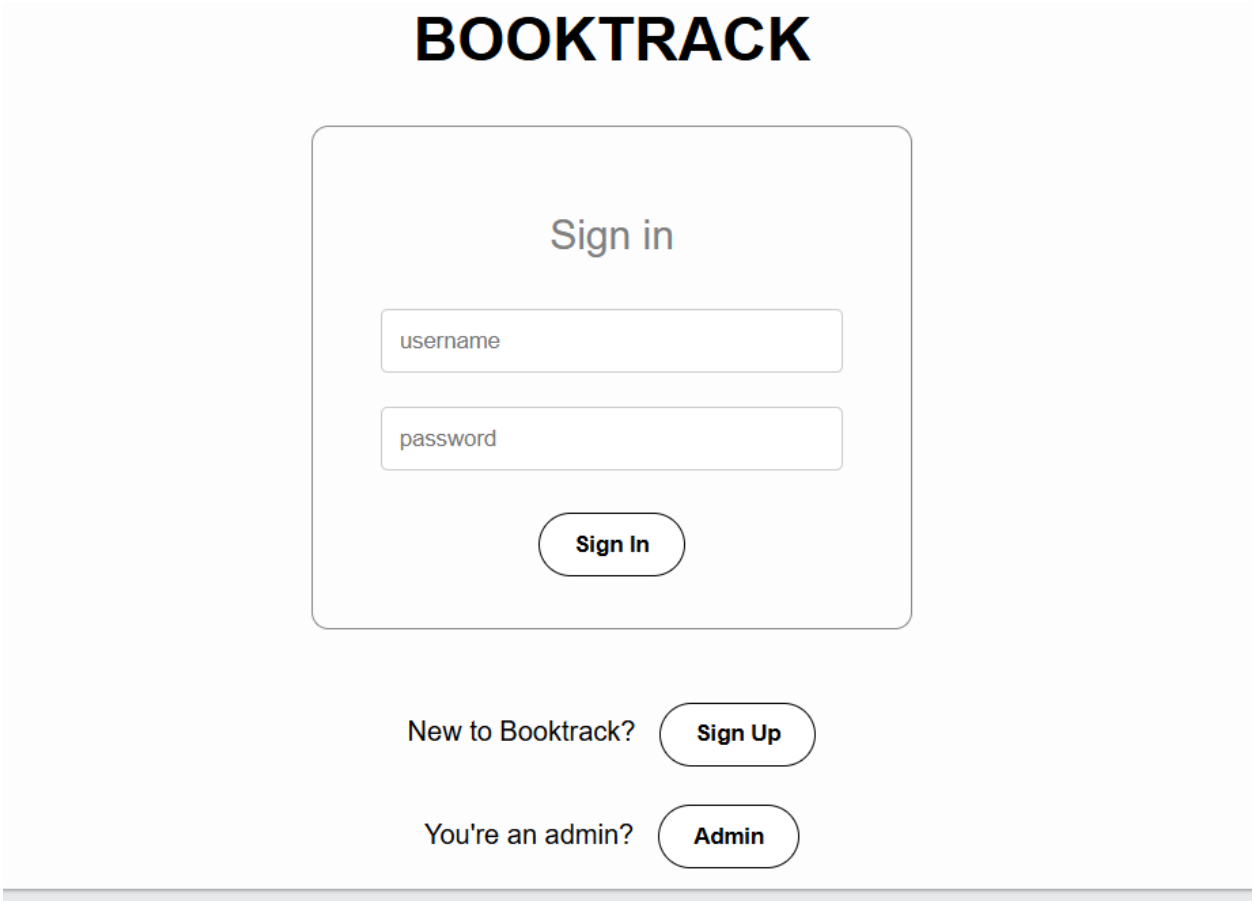


Figura 3: Prototipo della pagina di Sign in – MK#1

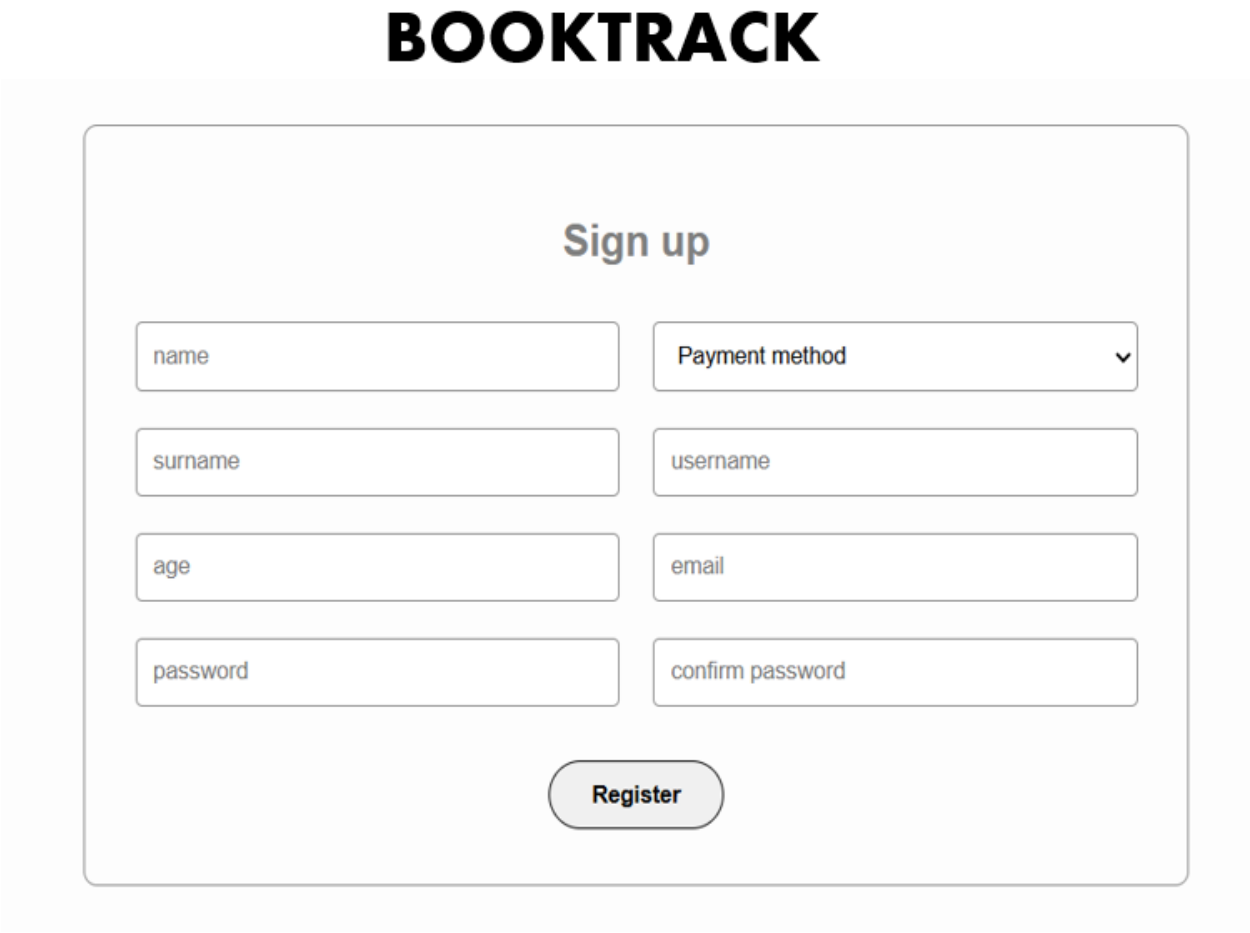
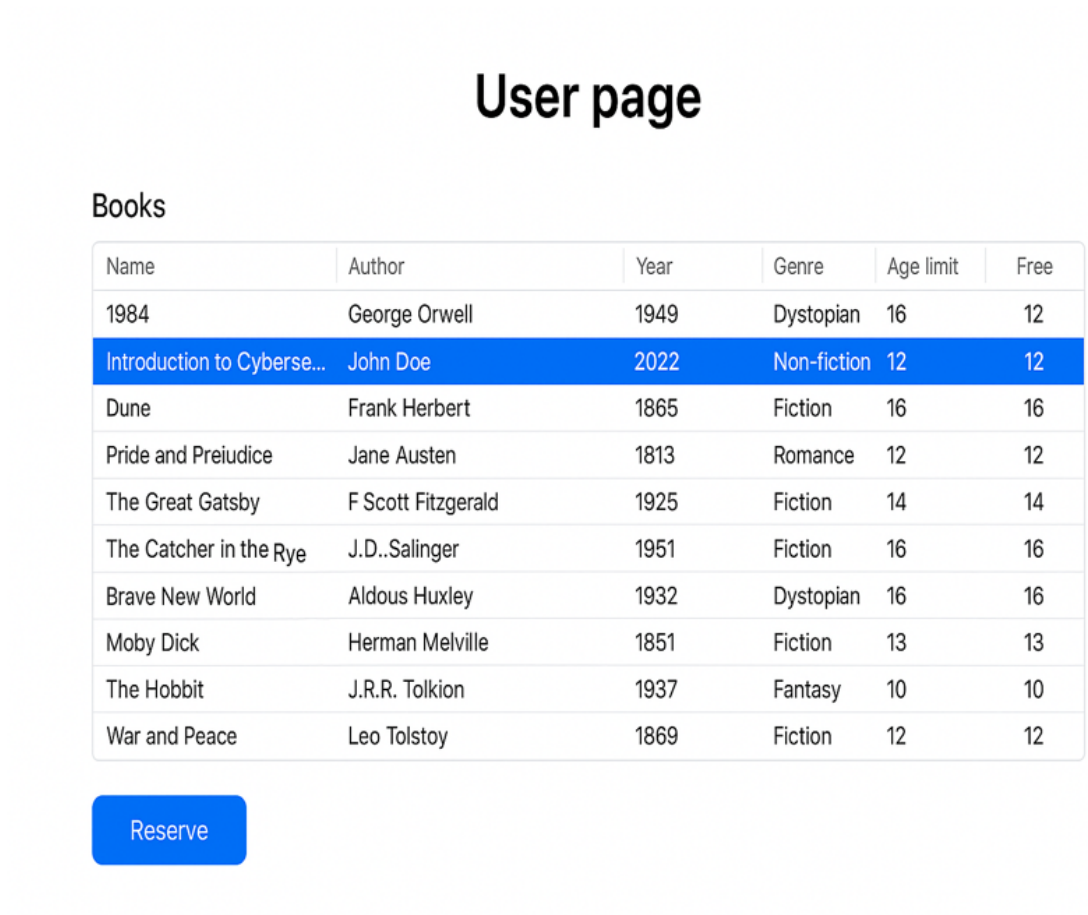


Figura 4: Prototipo della pagina di Registrazione – MK#2



User page

Books

Name	Author	Year	Genre	Age limit	Free
1984	George Orwell	1949	Dystopian	16	12
Introduction to Cyberse...	John Doe	2022	Non-fiction	12	12
Dune	Frank Herbert	1865	Fiction	16	16
Pride and Preiudice	Jane Austen	1813	Romance	12	12
The Great Gatsby	F Scott Fitzgerald	1925	Fiction	14	14
The Catcher in the Rye	J.D..Salinger	1951	Fiction	16	16
Brave New World	Aldous Huxley	1932	Dystopian	16	16
Moby Dick	Herman Melville	1851	Fiction	13	13
The Hobbit	J.R.R. Tolkion	1937	Fantasy	10	10
War and Peace	Leo Tolstoy	1869	Fiction	12	12

Reserve

Figura 5: Prototipo della pagina di prenotazione dei libri – MK#4

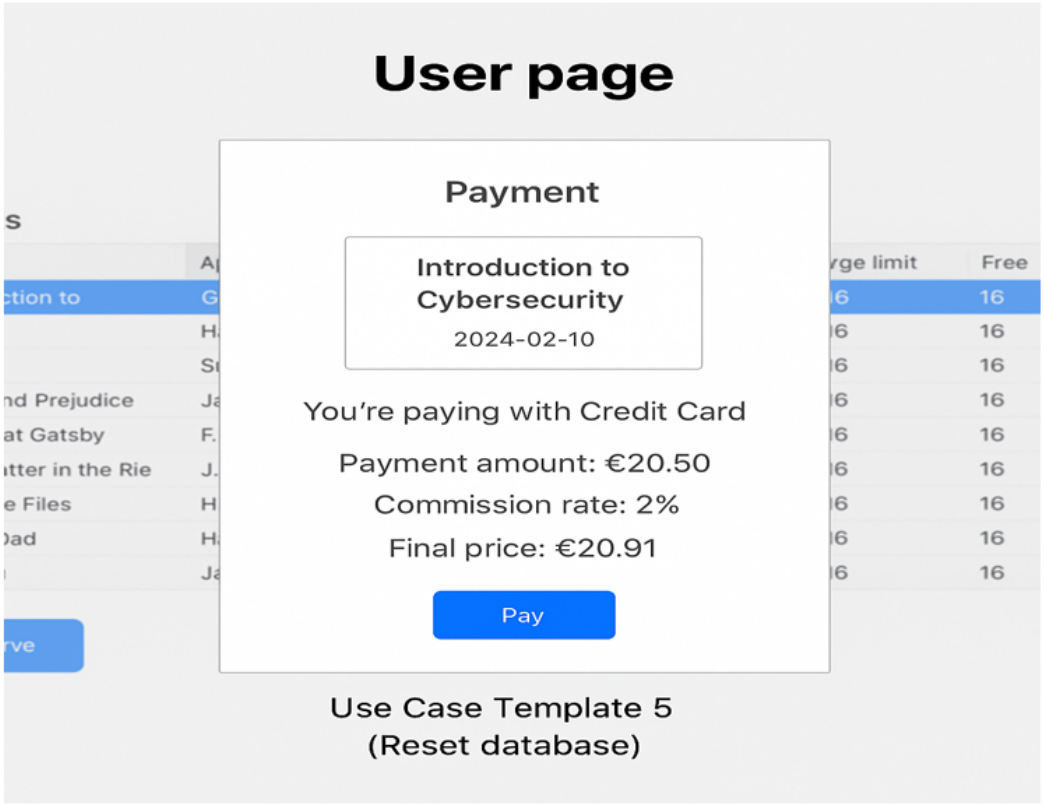


Figura 6: Prototipo della pagina di pagamento – MK#4

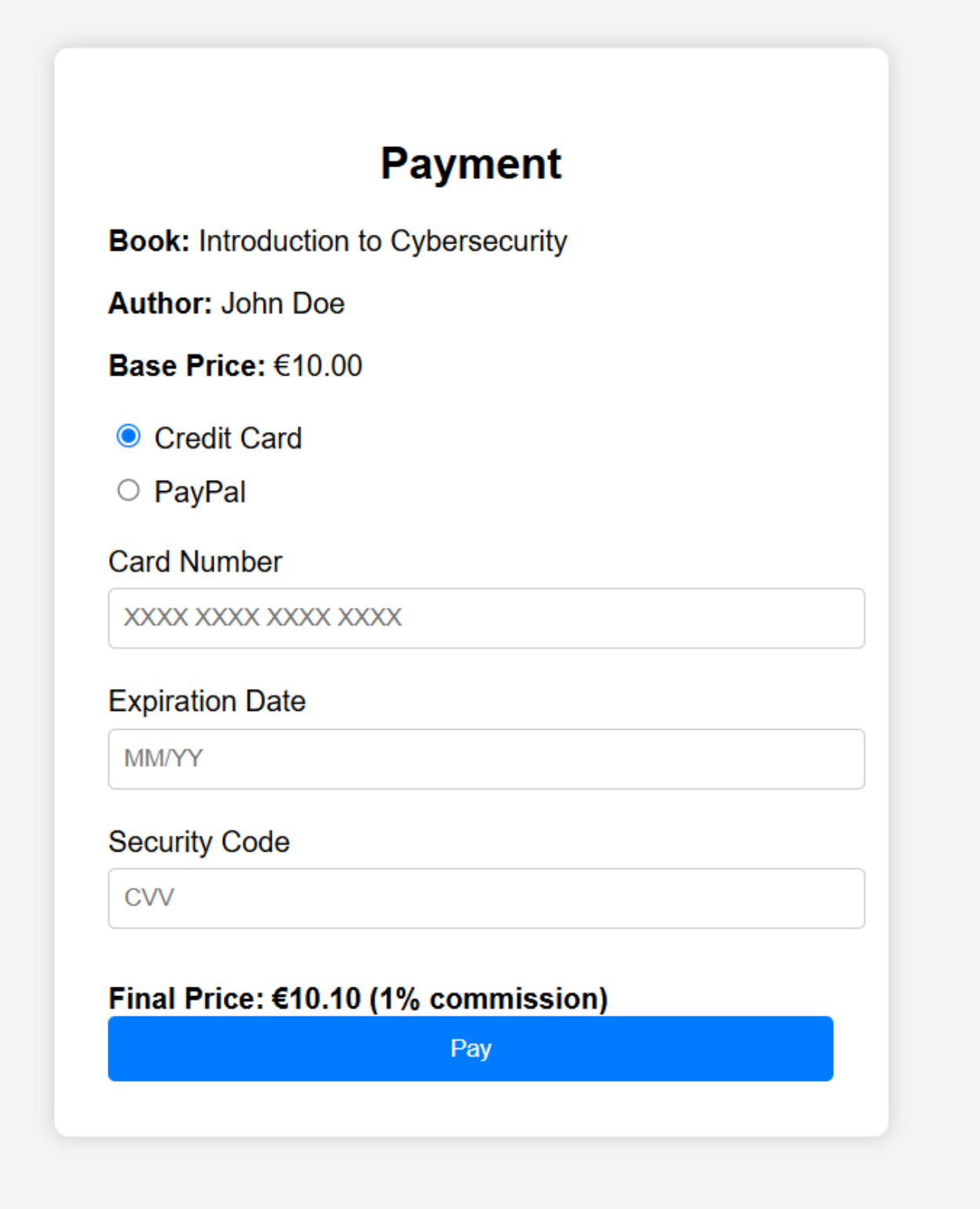


Figura 7: Prototipo della pagina di pagamento con credenziali personali

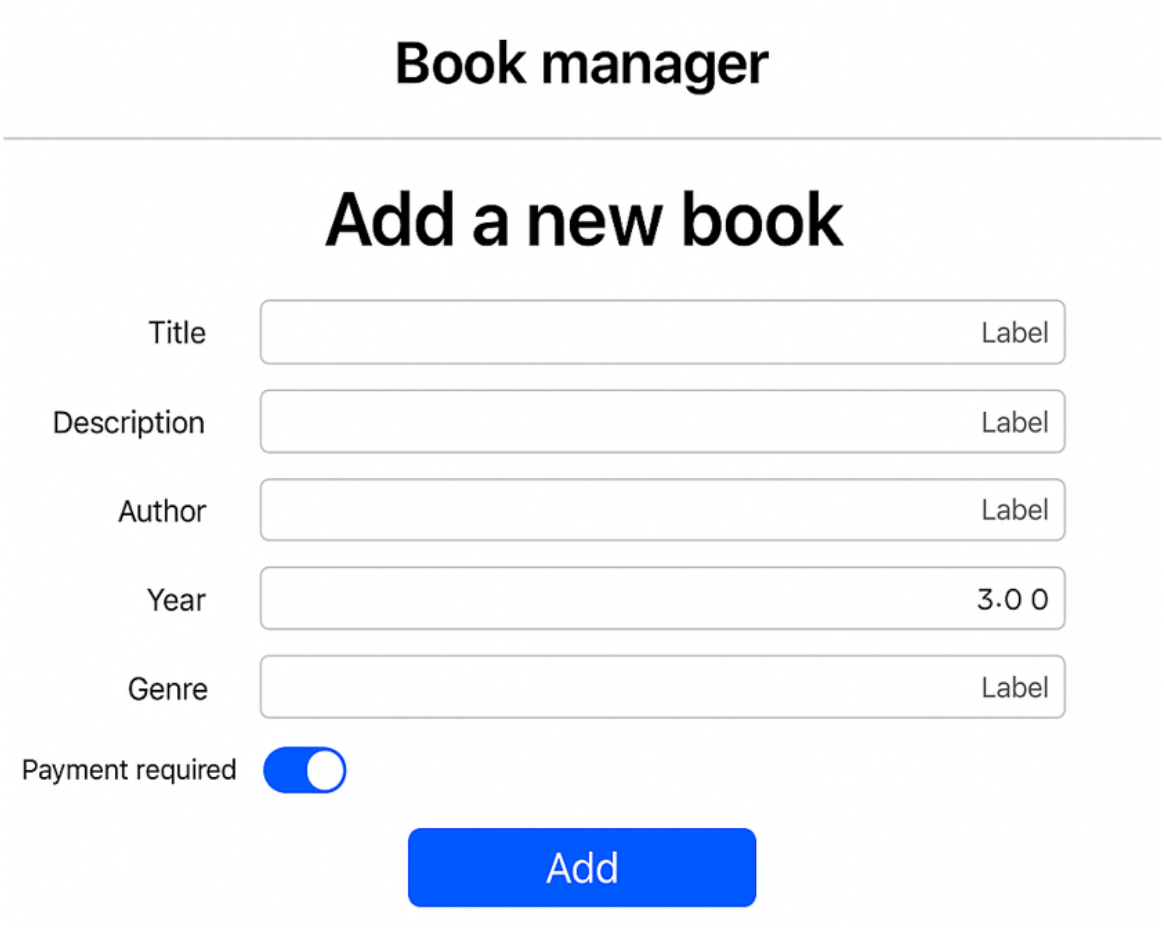


Figura 8: Prototipo della pagina di gestione dei libri (Admin)

2.5 Class Diagram

Vista la divisione modulare in tre principali *package* — `LibraryManagement`, `ORM` e `CoreEntities` — è stato realizzato un diagramma delle classi per ciascuno di essi.

LibraryManagement Il package `LibraryManagement` contiene le classi dedicate alla logica di business del sistema **BookTrack**. Le classi controller presenti gestiscono le operazioni principali del sistema lato utente e amministratore:

- **LoginController**
  Gestisce le funzionalità di accesso, registrazione e login amministratore.
- **UserBookPageController**
  Permette agli utenti di visualizzare i libri disponibili, cercare per tito-

lo/autore, effettuare prenotazioni e visualizzare o cancellare quelle esistenti.

- **UserReservationManagementController**
  Consente agli utenti di gestire e modificare le prenotazioni, richiedere estensioni e consultare i dettagli associati.

- **UserProfileController**
  Permette agli utenti di aggiornare le proprie informazioni personali e i metodi di pagamento (modifica dati, carta di credito, PayPal, ecc.).

- **AdminBookController**
  Gestisce tutte le operazioni sul catalogo libri da parte dell'amministratore (aggiunta, modifica, cancellazione), comprese le richieste utente.

- **AdminUserController**
  Consente all'amministratore di gestire gli utenti del sistema, con funzioni di visualizzazione, aggiunta e rimozione degli account.

- **AdminExtraController**
  Implementa le funzionalità di amministrazione avanzate, tra cui la modifica della password, il reset del database e la generazione di un set di dati predefinito.

Questa struttura segue il principio di **separazione delle responsabilità** (*Separation of Concerns – SoC*), in cui ogni controller è responsabile di un'area funzionale ben definita del sistema. Ciò garantisce manutenibilità, leggibilità e possibilità di estensione futura senza compromettere le altre componenti.

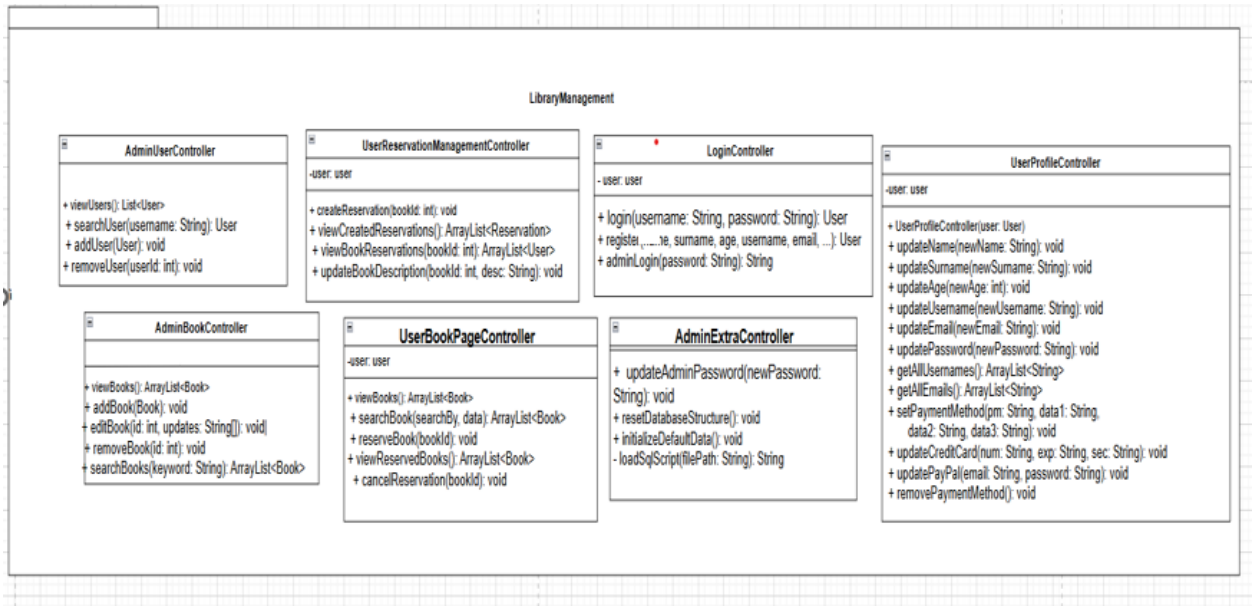


Figura 9: Diagramma delle classi del package `LibraryManagement`

CoreEntities Il package `CoreEntities` contiene le classi che rappresentano le entità fondamentali del sistema. Esse modellano gli oggetti reali e logici gestiti dall'applicativo, in particolare:

- **User**
  Rappresenta un utente del sistema che può esplorare, prenotare libri, scrivere recensioni, aggiornare il proprio profilo e i metodi di pagamento.

- **Book**
  Rappresenta un libro disponibile nel sistema, contenente informazioni dettagliate come titolo, autore, anno, genere, descrizione e disponibilità.

- **Reservation**
  Gestisce le prenotazioni effettuate dagli utenti, con informazioni relative alla data, durata, stato e utente associato.

- **CreditCard** e **PayPal**
  Rappresentano i metodi di pagamento associabili a un utente e utilizzabili per completare le prenotazioni.

- **PaymentStrategy**
  Implementa la strategia di pagamento seguendo il *design pattern Strategy*. Le classi derivate `CardPayment` e `PayPal` definiscono il comportamento specifico dei metodi di pagamento.

- **ReservationMapper**
  Implementa la gestione delle prenotazioni seguendo il *design pattern Mapper*, mappando i dati tra le entità del sistema e il database.

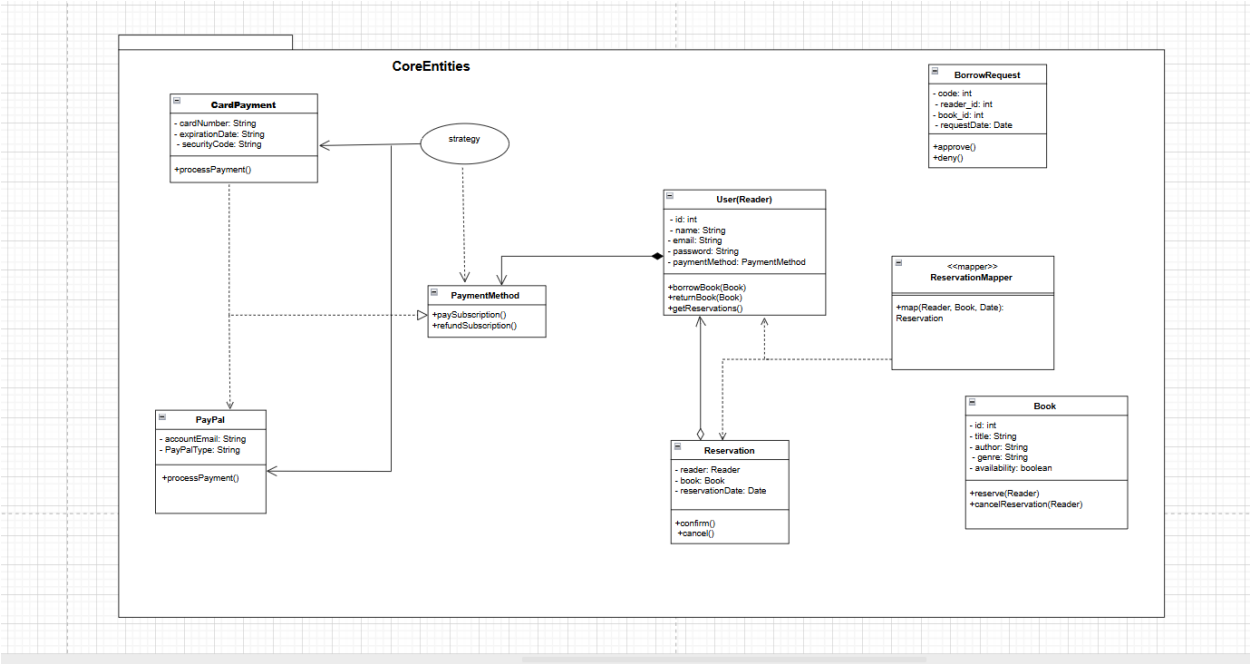


Figura 10: Diagramma delle classi del package `CoreEntities`

ORM (Object-Relational Mapping) Il package `ORM` contiene le classi responsabili della gestione dell'accesso ai dati e della loro persistenza nel database. Queste classi implementano le operazioni di tipo CRUD (*Create, Read, Update, Delete*) per ciascuna delle entità principali del sistema.

- **UserDAO**
  Gestisce le operazioni CRUD relative agli utenti: registrazione, autenticazione, aggiornamento profilo, eliminazione.

- **BookDAO**
  Gestisce le operazioni CRUD sui libri, inclusi l'inserimento nel catalogo, la modifica, la rimozione e la ricerca per titolo o autore.

- **ReservationDAO**
  Gestisce l'archiviazione, il recupero e la cancellazione delle prenotazioni effettuate dagli utenti.

- **CardPaymentDAO** e **PayPalDAO**
  Gestiscono le transazioni relative ai metodi di pagamento. Si occupano

della registrazione e verifica dei dati associati a carte di credito e account PayPal.

- **AdminDataAccess**
  Si occupa della gestione dei dati amministrativi, incluse le operazioni avanzate come il reset del database e la gestione degli account admin.

- **DBConnectionManager**
  Gestisce la connessione al database centralizzata, permettendo alle classi DAO di accedere al database in modo sicuro ed efficiente.

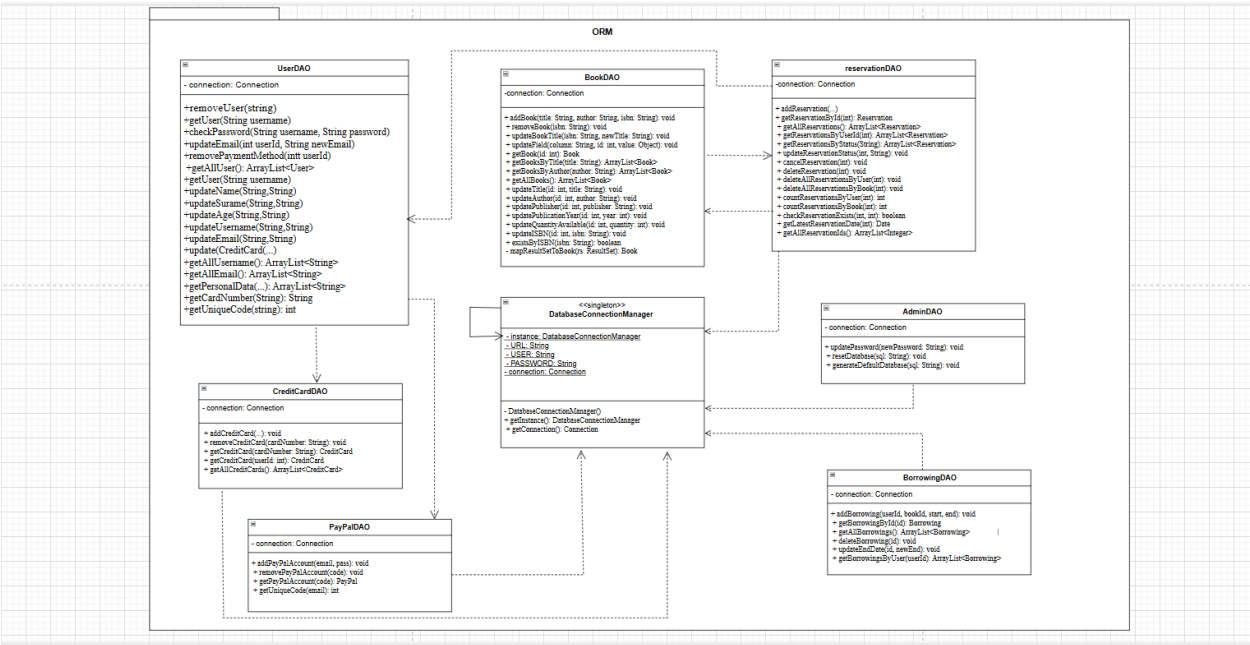


Figura 11: Diagramma delle classi del package `ORM`

2-5 ER Diagram e Modello Relazionale

Per quanto riguarda la progettazione del database, è stato realizzato un ER Diagram (Figura 12), successivamente ristrutturato (Figura 13) e infine trasformato nel modello relazionale definitivo (Figura 14).

Sono state effettuate alcune scelte progettuali significative, come ad esempio la ristrutturazione della generalizzazione **PaymentStrategy**. Infatti, sebbene questa generalizzazione fosse totale ed esclusiva (per scelta progettuale), si è comunque deciso di creare due tabelle distinte per le sottoclassi **CreditCard** e **PayPal**, favorendo così la leggibilità e la manutenibilità del database, a scapito di un leggero incremento nello spazio occupato e nella complessità delle query.

Inoltre, sono state modellate relazioni chiave come:

- **la prenotazione**(**Reservation**) effettuata da un utente;
- **Borrowing** il prestito di un libro con metodo di pagamento associato (**Borrowing**);
- **R2**: la strategia di pagamento utilizzata da ciascun utente (**R2**).

Sono state dunque definite le seguenti tabelle nel modello relazionale finale (Figura 14):

- **User**: rappresenta l'entità utente, che può prenotare o prendere in prestito dei libri.
- **Book**: rappresenta i libri disponibili nella libreria, con informazioni come titolo, autore, anno, prezzo, disponibilità, ecc.
- **Reservation**: risolve la relazione tra **User** e **Book** per la gestione delle prenotazioni.
- **Borrowing**: rappresenta la relazione tra utente e libro durante il prestito, incluso il metodo di pagamento.
- **CreditCard**: rappresenta l'entità **CreditCard** e risolve parte della generalizzazione **PaymentStrategy**.
- **PayPal**: rappresenta l'entità **PayPal** e completa la generalizzazione **PaymentStrategy**.

Questa struttura supporta una gestione modulare, scalabile e coerente dei dati e garantisce una buona integrità referenziale, facilitando anche l'estensione futura del sistema.

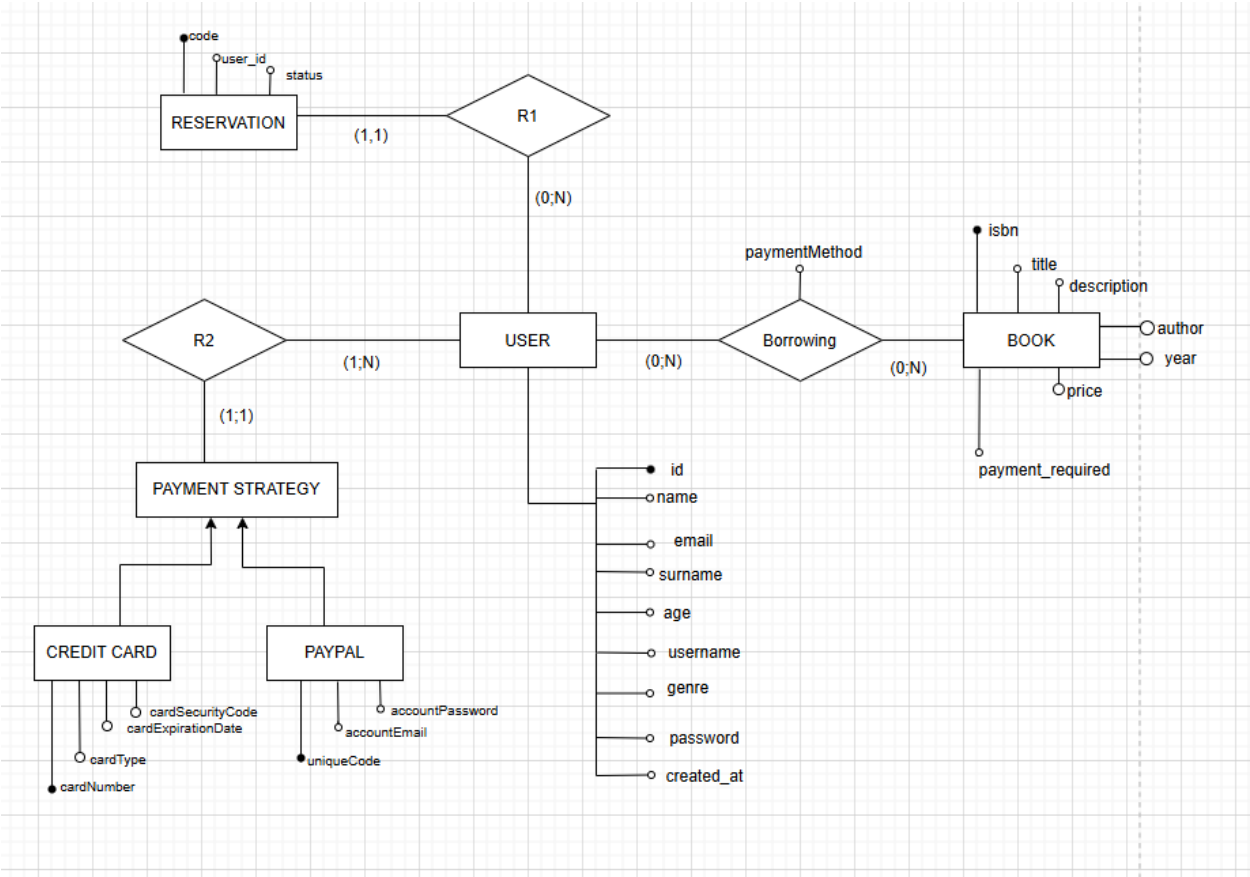


Figura 12: ER Diagram iniziale

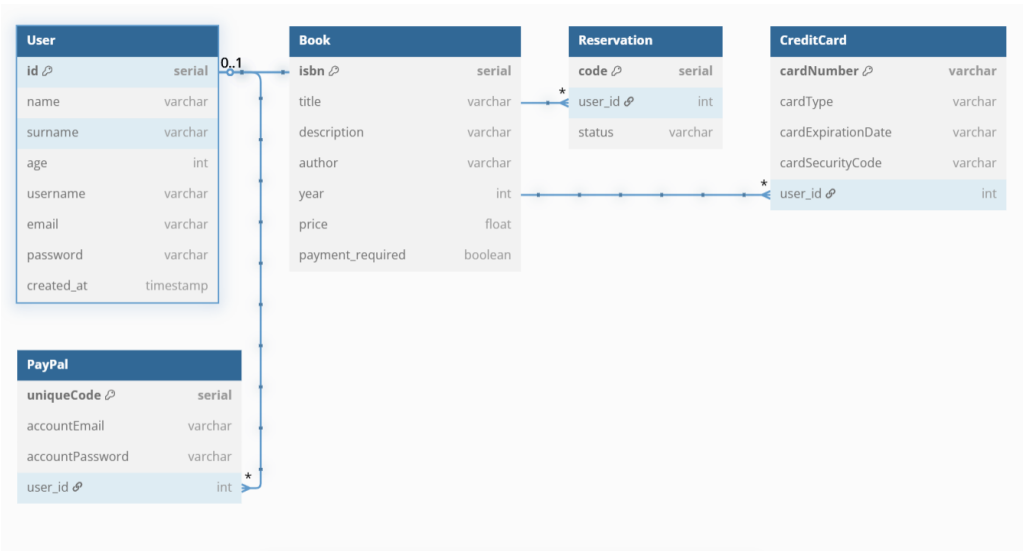


Figura 13: Schema ER ristrutturato

```
User (id, name, surname, age, username, email, password,
genre, created_at)
Book (isbn, title, description, author, year, price, pay-
ment_required)
Reservation (code, user_id, status)
Borrowing (user_id, isbn, paymentMethod)
CreditCard (cardNumber, cardType, cardExpirationDate,
cardSecurityCode)
PayPal (uniqueCode, accountEmail, accountPassword)
```

Snippet: modello relazionale

2.6 2-6 Navigation Diagram

Il seguente diagramma rappresenta le pagine principali del sistema **BookTrack** e le azioni che l'utente può compiere in ciascuna. Sono indicate anche le possibili navigazioni tra le sezioni, sia per l'utente normale che per l'amministratore.

Le principali pagine del sistema sono:

- **PAGINA INIZIALE (BOOKTRACK)**
  L'utente può:
  - Registrarsi (Sign up)
  - Effettuare il login
  - Accedere come admin
  - Uscire dal sistema
- **PAGINA UTENTE (USER PAGE)**
  Dopo il login, l'utente accede a:
  - UserBookPageController: visualizza libri, cerca un libro, effettua prenotazioni
  - UserReservationManagementController: gestisce le prenotazioni (creazione, modifica, cancellazione, richiesta estensione)
  - UserProfileController: modifica dati personali e metodi di pagamento
  - Logout

- **PROFILE EDITOR**
  Dalla sezione *"Profilo"*, l'utente accede a tre sottosezioni:
  - Dati personali: modifica nome, cognome, età
  - Dati login: modifica username, email, password
  - Dati di pagamento:
    * Aggiorna/mette/rimuove il metodo di pagamento
    * Aggiorna carta di credito
    * Aggiorna account PayPal
- **SEZIONE ADMIN (ADMIN PAGE)**
  Dopo il login come amministratore, l'admin può:
  - Gestire gli utenti (visualizzare, cercare, aggiungere, rimuovere)
  - Gestire i libri (aggiungere, modificare, rimuovere)
  - Gestire le richieste di modifica
  - Sezione extra
- **EXTRA (ADMINEXTRACONTROLLER)**
  Accessibile solo dall'admin, consente:
  - Modifica della password admin
  - Reset del database
  - Caricamento di dati di default

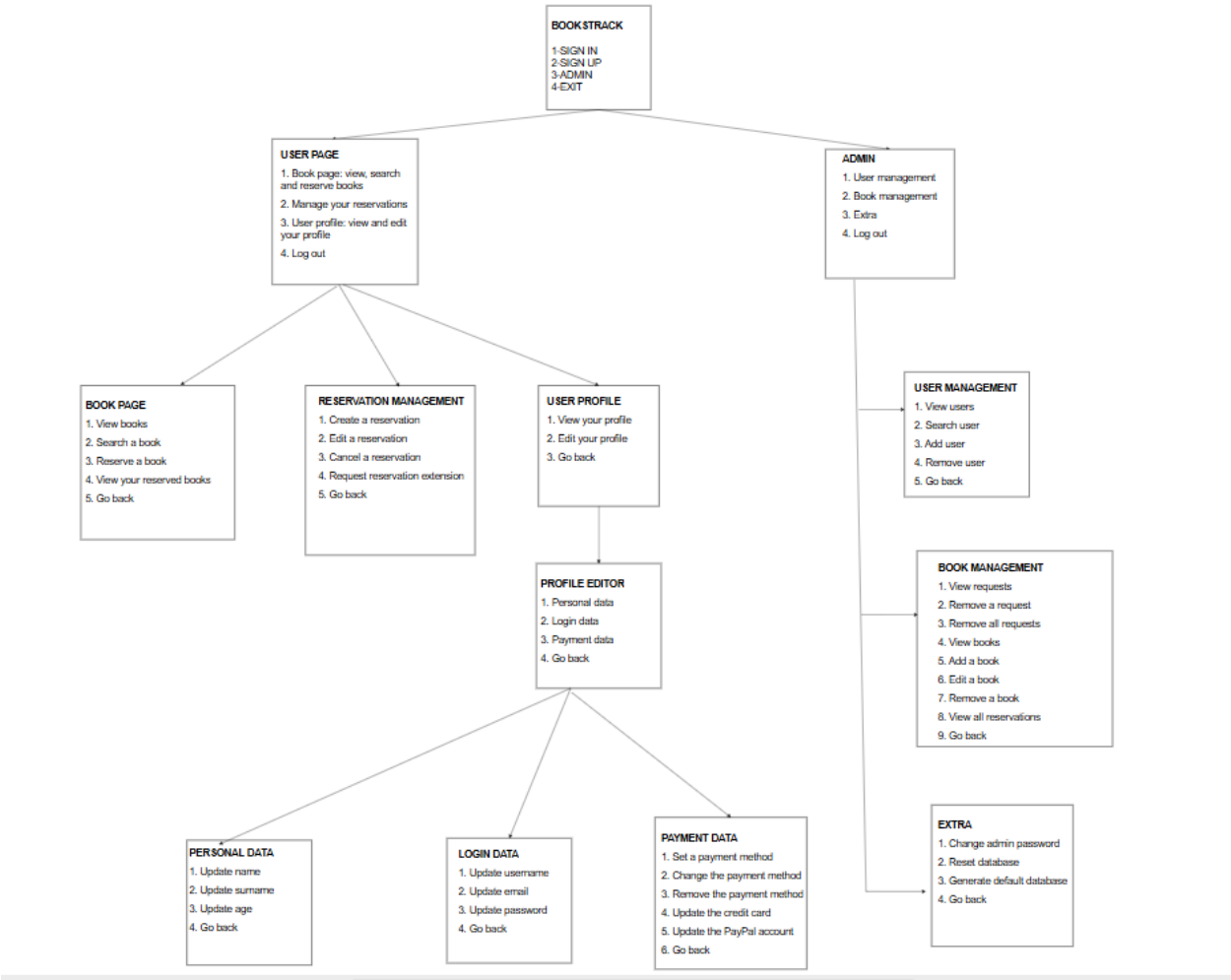


Figura 14: Navigation Diagram del sistema BookTrack

2.7 2.7 Struttura delle directory – BookTrack

La struttura delle directory del progetto è stata organizzata in modo da separare chiaramente i vari componenti logici e funzionali del sistema **BookTrack**. Tale organizzazione consente una gestione più ordinata del codice, della documentazione e dei test (vedi Figura 15).

Le directory e i file principali sono:

- **docs**: contiene la documentazione del progetto, inclusi i diagrammi UML, il file `.dbml` e i report tecnici.
- **lib/**: contiene le librerie esterne utilizzate dal sistema, come i driver JDBC per PostgreSQL o le librerie di testing (`JUnit`).

- **src/main**: contiene il codice sorgente dell'applicazione, organizzato per pacchetti (es. `BusinessLogic`, `DataAccess`, `Model`, `Utils`).
- **src/test**: contiene i test automatici sviluppati per verificare il corretto funzionamento delle componenti principali del sistema.
- `.idea`: contiene i file di configurazione dell'ambiente di sviluppo (IntelliJ IDEA).
- **.run**: contiene le configurazioni di esecuzione automatica per test o script specifici.
- **.gitignore**: specifica i file e le cartelle da ignorare nel controllo di versione Git.
- **misc.xml**, `modules.xml`, `vcs.xml`: file di configurazione del progetto generati automaticamente da IntelliJ.
- **booktrack_schema.sql.run.xml**: configurazione per l'esecuzione del file SQL relativo allo schema del database.

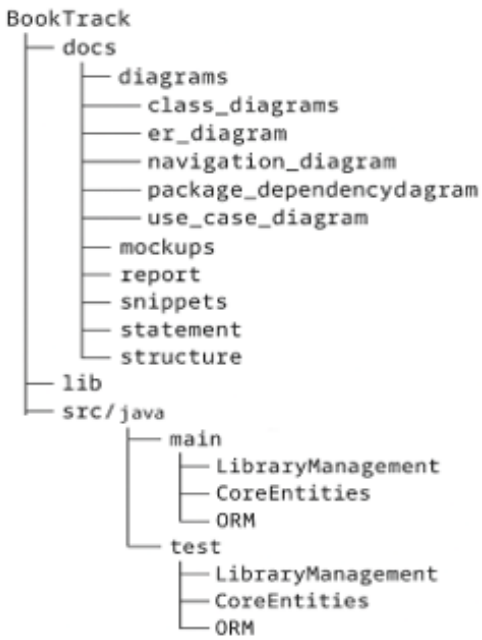


Figura 15: Struttura delle directory del progetto BookTrack

3 Implementazione

L'implementazione dei tre package `CoreEntities`, `LibraryManagement` e `ORM`, insieme a quella dell'interfaccia utente (se presente), è contenuta nella directory `src/main/java`, mentre i file relativi alla gestione e definizione del database si trovano nella directory `src/main/sql`.

3.1 CoreEntities

Come già accennato nella sezione 2.4, nel package `main.java.CoreEntities` (percorso: `src/main/java/CoreEntities`), sono state implementate le classi che rappresentano le entità fondamentali del sistema BookTrack.

3.1.1 User

La classe `User` rappresenta l'entità utente, ovvero un individuo registrato nel sistema. I campi della classe includono:

- `id`: identificatore univoco dell'utente
- `name`: nome
- `surname`: cognome
- `age`: età
- `username`: nome utente
- `email`: indirizzo email
- `password`: credenziale di accesso
- `created_at`: data di registrazione
- `paymentMethod`: istanza della strategia di pagamento implementata da `PaymentStrategy`

3.1.2 Book

La classe `Book` rappresenta un libro disponibile per la prenotazione nel sistema. I campi sono:

- `isbn`: identificatore univoco del libro

- `title`: titolo del libro
- `description`: descrizione
- `author`: autore
- `year`: anno di pubblicazione
- `price`: prezzo del libro
- `paymentRequired`: booleano che indica se il pagamento è obbligatorio

3.1.3 Reservation

La classe `Reservation` rappresenta una prenotazione di un libro da parte di un utente. Include i seguenti attributi:

- `code`: codice univoco della prenotazione
- `userId`: identificativo dell'utente che ha effettuato la prenotazione
- `bookId`: identificativo del libro prenotato
- `status`: stato attuale della prenotazione (es. attiva, annullata, completata)

3.1.4 Reservation e ReservationMapper

La classe `Reservation` rappresenta l'entità prenotazione, ovvero l'atto con cui un utente riserva un libro attraverso il sistema. I campi principali della classe sono:

- `user`: l'oggetto `User` che ha effettuato la prenotazione
- `book`: l'oggetto `Book` prenotato
- `paymentMethod`: l'istanza utilizzata per il pagamento, implementata tramite il design pattern `Strategy`

La classe `ReservationMapper` è stata progettata per mappare i dati della prenotazione, ovvero per convertire un'istanza di `Reservation` in un record del database, e viceversa. Essa collega logicamente l'entità `User` all'entità `Book` e gestisce la persistenza delle prenotazioni attraverso il layer ORM.

```java
package coreentities.librarymanagement;

public class ReservationMapper {

    public Reservation map(User user, Book book, String paymentMethod) {

        Reservation reservation = new Reservation();
        reservation.setUser(user);
        reservation.setBook(book);
        reservation.setPaymentMethod(paymentMethod);

        return reservation;

    }

}
```

Figura 16: Implementazione delle classi Reservation e ReservationMapper

3.1.5 PaymentStrategy, CreditCard e PayPal

Le classi `PaymentStrategy`, `CreditCard` e `PayPal` sono state implementate per gestire il comportamento dinamico dei pagamenti nel sistema BookTrack. Esse utilizzano il **design pattern Strategy**, che consente di scegliere a *runtime* la modalità di pagamento da utilizzare.

- `PaymentStrategy` è l'interfaccia principale e definisce i metodi comuni per tutte le strategie:
  - `pay()`: esegue il pagamento per una determinata prenotazione
  - `refund()`: effettua il rimborso
  - `getPaymentMethod()`: restituisce il nome del metodo di pagamento
  - `getPaymentData()`: restituisce i dati associati al pagamento (es. numero di carta, email PayPal, ecc.)
- `CreditCard` e `PayPal` sono le due classi concrete che implementano `PaymentStrategy`.
  Esse forniscono implementazioni specifiche dei metodi in modo coerente con il tipo di pagamento utilizzato.

Questa struttura offre flessibilità, permettendo di aggiungere in futuro nuovi metodi di pagamento (es. `BankTransfer`, `Crypto`, ecc.) senza modificare la logica della classe `User` o `Reservation`.

3.2 LibraryManagement

Come già accennato nella sezione 2.4, nel package `main.java.LibraryManagement` (percorso: `src/main/java/LibraryManagement`) sono state implementate le classi che gestiscono la logica di business del sistema BookTrack.

3.2.1 LoginController

Questa classe gestisce l'autenticazione e la registrazione degli utenti.
Offre i metodi:

- `login(username, password)`: verifica le credenziali nel database
- `register(...)`: registra un nuovo utente nel sistema, salvando le sue informazioni
- `adminLogin(...)`: autentica un amministratore

Per la comunicazione con il database utilizza la classe `UserDAO`.

3.2.2 UserBookPageController

Questa classe gestisce la visualizzazione dei libri disponibili per l'utente:

- `viewBooks()`: mostra tutti i libri disponibili alla prenotazione
- `searchBook(...)`: ricerca un libro secondo vari criteri (titolo, autore, anno, ecc.)
- `reserveBook(bookId)`: consente di prenotare un libro
- `viewReservedBooks()`: mostra tutti i libri prenotati dall'utente
- `cancelReservation(reservationId)`: annulla una prenotazione esistente

Comunica con il database tramite le classi `BookDAO` e `ReservationDAO`.

3.2.3 UserReservationManagementController

Tramite questa classe, l'utente può gestire le proprie prenotazioni:

- `createReservation(...)`: crea una nuova prenotazione
- `editReservation(...)`: modifica i dettagli di una prenotazione

- `deleteReservation(...)`: cancella una prenotazione
- `requestReservationExtension(...)`: invia una richiesta di estensione della durata al sistema
- `viewReservationDetails(...)`: visualizza i dettagli di una prenotazione specifica

Per la comunicazione con il database utilizza le classi `ReservationDAO` e `RequestDAO`.

3.2.4 UserProfileController

Con questa classe, l'utente può modificare i propri dati personali:

- `updateName()`, `updateSurname()`, `updateAge()`
- `updateUsername()`, `updateEmail()`, `updatePassword()`
- `updatePaymentMethod()`, `removePaymentMethod()`
- `updateCreditCard()`, `updatePayPal()`

Utilizza `UserDAO` per la comunicazione con il database.

3.2.5 AdminBookController

Questa classe gestisce le azioni dell'admin sui libri:

- `viewBooks()` e `searchBook()` per consultare i libri
- `addBook()`, `editBook()`, `removeBook()` per la gestione completa del catalogo

Gestisce anche le richieste di modifica da parte degli utenti tramite:

- `viewRequests()` e `removeRequest()`

Utilizza `BookDAO` e `RequestDAO`.

```java
public class AdminBookController {

    private final BookDAO bookDAO;

    public AdminBookController() {
        this.bookDAO = new BookDAO();
    }

    // 📚 Liste tous les livres
    public ArrayList<Book> viewBooks() throws SQLException {
        return bookDAO.getAllBooks();
    }

    // ➕ Ajoute un livre
    public void addBook(String title, String author, String publisher, int year, String isbn, int quantity) thro
        Book book = new Book(title, author, publisher, year, isbn, quantity);
        bookDAO.addBook(book);
        System.out.println("Livre ajouté avec succès.");
    }

    // ✏️ Modifie un livre
    public void editBook(int id, String[] updates) throws SQLException {
        if (updates[0] != null) bookDAO.updateTitle(id, updates[0]);
        if (updates[1] != null) bookDAO.updateAuthor(id, updates[1]);
        if (updates[2] != null) bookDAO.updatePublisher(id, updates[2]);
        if (updates[3] != null) bookDAO.updatePublicationYear(id, Integer.parseInt(updates[3]));
        if (updates[4] != null) bookDAO.updateISBN(id, updates[4]);
        if (updates[5] != null) bookDAO.updateQuantityAvailable(id, Integer.parseInt(updates[5]));
    }
```

Figura 17: implementazione class AdminBookController

3.2.6 AdminUserController

Questa classe gestisce la gestione utenti da parte dell'admin:

- `viewUsers()` e `searchUser()` per la visualizzazione
- `addUser()` e `removeUser()` per aggiunta e cancellazione di utenti

Comunica con `UserDAO`.

```java
public class AdminUserController {

    private final UserDAO userDAO;

    public AdminUserController() {
        this.userDAO = new UserDAO();
    }

    public List<User> viewUsers() throws SQLException {
        return userDAO.getAllUsers();
    }

    public User searchUser(String username) throws SQLException {
        return userDAO.getUserByUsername(username);
    }

    public void addUser(String name, String surname, int age, String username, String email, String password,
                        String paymentMethod, String cardNumber, String expirationDate, String securityCode)
            throws SQLException {

        User user = new User(name, surname, age, username, email, password,
                paymentMethod, cardNumber, expirationDate, securityCode);
        userDAO.addUser(user);
        System.out.println("✅ Utilisateur ajouté avec succès.");
    }

    public void removeUser(int userId) throws SQLException {
        userDAO.removeUser(userId);
        System.out.println("❌ Utilisateur supprimé.");
    }
}
```

Figura 18: implementazione class AdminUserController

3.2.7 AdminExtraController

Questa classe fornisce funzionalità amministrative aggiuntive:

- `changePassword()`: modifica della password admin
- `resetDatabase()`: reset totale del database
- `generateDefaultDatabase()`: genera un set di dati predefinito

Per tutte queste operazioni, utilizza la classe `AdminDAO`.

```java
public class AdminExtraController {

    /**
     * Aggiorna la password dell'account admin nel database.
     * @param newPassword la nuova password da impostare
     * @throws SQLException
     * @throws ClassNotFoundException
     */
    public void updateAdminPassword(String newPassword) throws SQLException, ClassNotFoundException {
        AdminDAO adminDAO = new AdminDAO();
        adminDAO.updatePassword(newPassword);
    }

    /**
     * Esegue il reset completo del database eliminando e ricreando le tabelle secondo lo script reset.sql.
     * @throws SQLException
     * @throws ClassNotFoundException
     */
    public void resetDatabaseStructure() throws SQLException, ClassNotFoundException {
        String sql = loadSqlScript("src/main/sql/reset.sql");

        if (sql != null) {
            AdminDAO adminDAO = new AdminDAO();
            adminDAO.resetDatabase(sql);
        }
    }
```

Figura 19: implementazione class AdminExtraController

3.3 Object-Relational Mapping

Come già accennato nella sezione 2.4, nel package `main.java.ORM` (percorso: `src/main/java/ORM`) sono state implementate le classi che si occupano dell'Object-Relational Mapping, ovvero delle operazioni di lettura, scrittura, aggiornamento e cancellazione dei dati nel database relazionale utilizzato da BookTrack.

3.3.1 DatabaseConnectionManager

La classe `DatabaseConnectionManager` gestisce la connessione al database per tutte le altre classi DAO attraverso il metodo `getConnection()`.
Contiene inoltre i parametri di configurazione della connessione, ovvero:

- `DB_URL`
- `USERNAME`

- PASSWORD

La classe è implementata come singleton, in modo da garantire l'unicità dell'istanza e prevenire conflitti o aperture multiple non gestite della connessione.

```java
public class DatabaseConnectionManager {

    private static DatabaseConnectionManager instance;
    private static final String URL = "jdbc:postgresql://localhost:5432/booktrack_db"; // ← Cambia con il tuo da
    private static final String USER = "postgres"; // ← Cambia con il tuo utente DB
    private static final String PASSWORD = "password"; // ← Cambia con la tua password DB

    private Connection connection;

    private DatabaseConnectionManager() throws SQLException, ClassNotFoundException {
        Class.forName("org.postgresql.Driver");
        this.connection = DriverManager.getConnection(URL, USER, PASSWORD);
    }

    /**
     * Ottieni l'istanza singleton del gestore di connessione
     */
    public static DatabaseConnectionManager getInstance() throws SQLException, ClassNotFoundException {
        if (instance == null || instance.getConnection().isClosed()) {
            instance = new DatabaseConnectionManager();
        }
        return instance;
    }

    /**
     * Restituisce la connessione al database
     */
    public Connection getConnection() {
        return this.connection;
    }
}
```

Figura 20: implementazione class DatabaseConnectionManager

3.3.2 UserDAO e AdminDAO

Queste due classi si occupano della gestione dei dati rispettivamente degli utenti e dell'amministratore.

- **UserDAO** contiene metodi quali:
  - `addUser(User user)`
  - `removeUser(int userId)`

- `getUser(String username)`
- `checkPassword(String username, String password)`
- `updateEmail(int userId, String newEmail)`
- `removePaymentMethod(int userId)`

Per la rimozione di metodi di pagamento, `UserDAO` si appoggia a `CreditCardDAO` e `PayPalDAO`.

```
public class UserDAO {

    private Connection connection;

    public UserDAO() {
        try {
            this.connection = DatabaseConnectionManager.getInstance().getConnection();
        } catch (SQLException | ClassNotFoundException e) {
            System.err.println("Errore nella connessione: " + e.getMessage());
        }
    }

    public void addUser(String username, String email, String password) throws SQLException {
        String sql = "INSERT INTO users (username, email, password) VALUES (?, ?, ?)";
        try (PreparedStatement ps = connection.prepareStatement(sql)) {
            ps.setString(1, username);
            ps.setString(2, email);
            ps.setString(3, password);
            ps.executeUpdate();
            System.out.println("Utente aggiunto correttamente.");
        }
    }

    public User getUserById(int id) throws SQLException {
        String sql = "SELECT * FROM users WHERE id = ?";
        try (PreparedStatement ps = connection.prepareStatement(sql)) {
            ps.setInt(1, id);
            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) {
                    return new User(
                            rs.getInt("id"),
                            rs.getString("username"),
```

```
                                rs.getString("password"),
                                rs.getInt("credit_card_id"),
                                rs.getInt("paypal_id")
                        );
                    }
                }
            }
            return null;
        }

        public User getUserByUsername(String username) throws SQLException {
            String sql = "SELECT * FROM users WHERE username = ?";
            try (PreparedStatement ps = connection.prepareStatement(sql)) {
                ps.setString(1, username);
                try (ResultSet rs = ps.executeQuery()) {
                    if (rs.next()) {
                        return new User(
                                rs.getInt("id"),
                                rs.getString("username"),
                                rs.getString("email"),
                                rs.getString("password"),
                                rs.getInt("credit_card_id"),
                                rs.getInt("paypal_id")
                        );
                    }
                }
            }
            return null;
        }
```

Figura 21: Costruttore classe UserDao con metodi addUser,getUserById,updateEmail

- **AdminDAO** fornisce solo i metodi necessari per supportare le operazioni della classe `AdminExtraController`, come:
  - `changePassword(String oldPass, String newPass)`
  - `resetDatabase()`
  - `generateDefaultData()`

```java
public class AdminDAO {

    private Connection connection;

    public AdminDAO() {
        try {
            this.connection = ConnectionManager.getInstance().getConnection();
        } catch (SQLException | ClassNotFoundException e) {
            System.err.println("Error: " + e.getMessage());
        }
    }

    public void updatePassword(String newPassword) throws SQLException {
        String sql = String.format("UPDATE \"User\" SET password = '%s' WHERE username = 'ADMIN'", newPassword);
        PreparedStatement preparedStatement = null;

        try {
            preparedStatement = connection.prepareStatement(sql);
            preparedStatement.executeUpdate();
            System.out.println("Admin password updated successfully.");
        } catch (SQLException e) {
            System.err.println("Error updating admin password: " + e.getMessage());
        } finally {
            if (preparedStatement != null) {
                preparedStatement.close();
            }
        }
    }
```

Figura 22: class AdminDAO con metodo updatepassword

```java
public class AdminDAO {

    public void resetDatabase(String sql) throws SQLException {
        PreparedStatement preparedStatement = null;

        try {
            preparedStatement = connection.prepareStatement(sql);
            preparedStatement.executeUpdate();
            System.out.println("\nBookTrack database reset successfully.");
        } catch (SQLException e) {
            System.err.println("Error resetting database: " + e.getMessage());
        } finally {
            if (preparedStatement != null) {
                preparedStatement.close();
            }
        }
    }

    public void generateDefaultDatabase(String sql) throws SQLException {
        PreparedStatement preparedStatement = null;

        try {
            preparedStatement = connection.prepareStatement(sql);
            preparedStatement.executeUpdate();
            System.out.println("BookTrack default database generated successfully.");
        } catch (SQLException e) {
            System.err.println("Error generating default database: " + e.getMessage());
        } finally {
            if (preparedStatement != null) {
                preparedStatement.close();
            }
        }
```

Figura 23: class AdminDAO con metodo resetdtbase genratedatabase

3.3.1 BookDAO

La classe `BookDAO` si occupa della gestione dei dati dei libri presenti nel sistema.
Offre metodi per:

- `addBook(Book book)`
- `removeBook(int bookId)`
- `getBook(int bookId)`
- `getBooksByTitle(String title)`

- getBooksByAuthor(String author)
- updateBookDetails(int bookId, Book updatedBook)

```java
public class BookDAO {

    private Connection connection;

    public BookDAO() {
        try {
            this.connection = ConnectionManager.getInstance().getConnection();
        } catch (SQLException | ClassNotFoundException e) {
            System.err.println("Error initializing BookDAO: " + e.getMessage());
        }
    }

    public void addBook(String title, String author, String isbn) throws SQLException {
        String sql = "INSERT INTO \"Book\" (title, author, isbn) VALUES (?, ?, ?)";
        try (PreparedStatement ps = connection.prepareStatement(sql)) {
            ps.setString(1, title);
            ps.setString(2, author);
            ps.setString(3, isbn);
            ps.executeUpdate();
            System.out.println("Book added successfully.");
        }
    }

    public void removeBook(String isbn) throws SQLException {
        String sql = "DELETE FROM \"Book\" WHERE isbn = ?";
        try (PreparedStatement ps = connection.prepareStatement(sql)) {
            ps.setString(1, isbn);
            ps.executeUpdate();
            System.out.println("Book removed successfully.");
        }
    }
```

Figura 24: class BookDAO con metodo addBook removeBook

Serve come interfaccia per il catalogo libri utilizzato da utenti e admin.

3.3.2 ReservationDAO e RequestDAO

- ReservationDAO gestisce le operazioni relative alle prenotazioni di libri. Include metodi come:
  - addReservation(Reservation r)
  - removeReservation(int reservationId)

  - `getReservationsByUser(int userId)`
  - `getReservationsByBook(int bookId)`
  - `updateReservationStatus(int reservationId, String newStatus)`

- `RequestDAO` si occupa delle richieste di modifica o estensione fatte dagli utenti.
  Offre metodi come:
  - `addRequest(Request req)`
  - `removeRequest(int requestId)`
  - `removeAllRequestsByUser(int userId)`
  - `getRequestsByStatus(String status)`

3.3.3 CreditCardDAO e PayPalDAO

Queste classi gestiscono i dati dei metodi di pagamento associati agli utenti:

- **CreditCardDAO**:
  - `addCreditCard(int userId, String cardNumber, String expiry, String cvv)`
  - `removeCreditCard(int userId)`
  - `getCreditCard(int userId)`

```java
public class CreditCardDAO {

    private Connection connection;

    public CreditCardDAO() {
        try {
            this.connection = ConnectionManager.getInstance().getConnection();
        } catch (SQLException | ClassNotFoundException e) {
            System.err.println("Error: " + e.getMessage());
        }
    }

    public void addCreditCard(String cardNumber, String cardExpirationDate, String cardSecurityCode) throws SQLE
        String sql = "INSERT INTO \"CreditCard\" (cardNumber, cardExpirationDate, cardSecurityCode) VALUES (?, ?
        try (PreparedStatement preparedStatement = connection.prepareStatement(sql)) {
            preparedStatement.setString(1, cardNumber);
            preparedStatement.setString(2, cardExpirationDate);
            preparedStatement.setString(3, cardSecurityCode);
            preparedStatement.executeUpdate();
            System.out.println("Credit Card added successfully.");
        }
    }

    public void removeCreditCard(String cardNumber) throws SQLException {
        String sql = "DELETE FROM \"CreditCard\" WHERE cardNumber = ?";
        try (PreparedStatement preparedStatement = connection.prepareStatement(sql)) {
            preparedStatement.setString(1, cardNumber);
            preparedStatement.executeUpdate();
            System.out.println("Credit Card removed successfully.");
        }
    }
```

Figura 25: class CreditCardDAO

- **PayPalDAO**:
  - `addPayPalAccount(int userId, String email)`
  - `removePayPalAccount(int userId)`
  - `getPayPalAccount(int userId)`

Entrambe sono frequentemente utilizzate da `UserDAO` nei metodi di aggiornamento o cancellazione del metodo di pagamento.

```java
public class PayPalDAO {

    private Connection connection;

    public PayPalDAO() {
        try {
            this.connection = ConnectionManager.getInstance().getConnection();
        } catch (SQLException | ClassNotFoundException e) {
            System.err.println("Error: " + e.getMessage());
        }
    }

    public void addPayPalAccount(String accountEmail, String accountPassword) throws SQLException {

        String sql = String.format("INSERT INTO \"PayPal\" (accountEmail, accountPassword) VALUES ('%s', '%s')",
                accountEmail, accountPassword);

        PreparedStatement preparedStatement = null;

        try {
            preparedStatement = connection.prepareStatement(sql);
            preparedStatement.executeUpdate();
            System.out.println("PayPal account added successfully.");
        } catch (SQLException e) {
            System.err.println("Error: " + e.getMessage());
        } finally {
            if (preparedStatement != null) {
                preparedStatement.close();
            }
        }
```

Figura 26: class PayPalDAO

3.4 Database

Come già accennato nella sezione 2.5, il database è stato progettato in modo da rispettare le specifiche funzionali del sistema BookTrack. Sono state create le tabelle `User`, `CreditCard`, `PayPal`, `Book`, `Borrowing` e `Reservation`, come illustrato nella Figura 27.

I file relativi al database sono contenuti nella cartella `src/main/sql` e i principali sono:

- `reset_bookstrack_data.sql`: cancella le tabelle esistenti tramite il comando `DROP TABLE IF EXISTS` e le ricrea secondo la struttura definita nel modello relazionale.

- `default.sql`: popola le tabelle con dati di default per consentire il test e la validazione del sistema in fase di sviluppo.

La progettazione del database ha seguito criteri di chiarezza, coerenza e modularità, prevedendo anche la gestione dei metodi di pagamento tramite una generalizzazione `PaymentStrategy` (con sottoclassi `CreditCard` e `PayPal`) modellata separatamente in due tabelle, per semplificare manutenzione e lettura, anche a discapito di un piccolo aumento dell'occupazione in memoria.

```sql
-- Carta di Credito
CREATE TABLE "CreditCard" (
                            cardNumber VARCHAR(50) PRIMARY KEY,
                            cardType VARCHAR(50),
                            cardExpirationDate VARCHAR(50) NOT NULL,
                            cardSecurityCode VARCHAR(50) NOT NULL
);

-- Conto PayPal
CREATE TABLE "PayPal" (
                        uniqueCode SERIAL PRIMARY KEY,
                        accountEmail VARCHAR(100) NOT NULL UNIQUE,
                        accountPassword VARCHAR(100) NOT NULL
);

-- Utente
CREATE TABLE "User" (
                      id SERIAL PRIMARY KEY,
                      name VARCHAR(50),
                      surname VARCHAR(50),
                      age INTEGER CONSTRAINT bt_age_positive CHECK (age >= 0 AND age < 150),
                      username VARCHAR(50) NOT NULL UNIQUE,
                      email VARCHAR(100) NOT NULL UNIQUE,
                      password VARCHAR(100) NOT NULL,
                      creditCard VARCHAR(50) UNIQUE,
                      payPal INTEGER UNIQUE,
                      FOREIGN KEY (creditCard) REFERENCES "CreditCard"(cardNumber),
                      FOREIGN KEY (payPal) REFERENCES "PayPal"(uniqueCode),
                      created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Figura 27: ricreazione e rimozione della tabella user nel file reset$_{b}ookstrack_{d}ata.sqlpaypalecreditcard$

3.5 Interfaccia a Riga di Comando (CLI)

Come introdotto nella sezione 1.2, è stata realizzata un'interfaccia a riga di comando per il sistema BookTrack, implementata nel file `Main.java` (il percorso è: `src/main/java/Main.java`).

L'utente può navigare tra le varie pagine e svolgere le principali funzionalità del sistema inserendo i comandi indicati di volta in volta.
Ad esempio, nella pagina iniziale, l'utente ha la possibilità di:

- accedere al proprio account inserendo il comando `1`,
- registrarsi come nuovo lettore inserendo il comando `2`,
- autenticarsi come amministratore con il comando `3`,
- oppure uscire dal programma con il comando `4`.

Ogni "pagina" dell'interfaccia è gestita tramite un ciclo `do-while` e una struttura `switch-case`, che interpreta e gestisce le scelte dell'utente.

Le funzionalità vere e proprie sono delegate a metodi specifici delle classi contenute nel package `main.java.BusinessLogic`, ciascuna responsabile di un determinato aspetto del sistema (prenotazione libri, gestione profilo, operazioni admin, ecc.).

La navigazione tra le pagine è ottenuta tramite semplici chiamate ricorsive o dirette ai metodi della classe `Main`, garantendo così un flusso di controllo ben definito e leggibile.

```java
public class Main {

    public static void main(String[] args) throws SQLException, ClassNotFoundException {
        gestisciAccesso();
    }

    public static void gestisciAccesso() throws SQLException, ClassNotFoundException {
        Scanner scanner = new Scanner(System.in);
        LoginController loginController = new LoginController();

        String input;
        do {
            System.out.println("""
                BENVENUTO IN BOOKTRACK
                1. Accedi
                2. Registrati
                3. Area Admin
                4. Esci
            """);

            input = scanner.nextLine();
            switch (input) {
                case "1" -> {
                    System.out.print("Username: ");
                    String username = scanner.nextLine();
                    System.out.print("Password: ");
                    String password = scanner.nextLine();

                    User user = loginController.login(username, password);
                    if (user != null) gestisciUtente(user);
                }
```

Figura 28: implementazione pagina iniziale e sign in

4 Testing

Sono stati realizzati test per verificare il corretto funzionamento del sistema BookTrack.
In particolare, sono stati effettuati:

- test funzionali su diversi metodi del package `LibraryManagement`, volti a verificare la corretta esecuzione delle funzionalità principali del sistema (autenticazione, prenotazione, gestione profilo, ecc.);
- test strutturali su metodi significativi del package `CoreEntities`, con l'obiettivo di validare il corretto comportamento delle classi `User`, `Book`, `Reservation`, `CreditCard`, `PayPal`, ecc.

Per quanto riguarda il package `ORM` (ovvero i DAO), non sono stati realizzati test unitari specifici, in quanto la correttezza dei metodi è stata verificata indirettamente attraverso i test applicati alla logica di business e alle operazioni che fanno uso di accesso al database.

4.1 LibraryManagement Test

Per garantire l'affidabilità e la correttezza del sistema BookTrack, sono stati sviluppati test funzionali per i principali controller presenti nel package `main.java.LibraryManagement`.
I test verificano il comportamento dei metodi in scenari realistici, sfruttando un database popolato con dati di default.

Le classi di test implementate sono:

- `LoginControllerTest`
  - `loginTest()`: verifica il login con credenziali corrette e errate.
  - `registerTest()`: controlla la corretta registrazione di nuovi utenti.
  - `adminLoginTest()`: testa l'autenticazione dell'amministratore con credenziali valide.
- `UserBookPageControllerTest`
  - `reserveBookTest()`: verifica che un utente possa prenotare un libro disponibile.
  - `cancelReservationTest()`: controlla la rimozione corretta di una prenotazione.

- `UserBookManagementControllerTest`
  - `addBookTest()`: testa la creazione di un nuovo libro nel sistema da parte dell'utente.
  - `requestToRemoveBookTest()`: simula una richiesta di cancellazione del libro già presente.
- `UserProfileControllerTest`
  - `updatePaymentMethodTest()`: verifica l'aggiornamento del metodo di pagamento (es. da PayPal a carta di credito).
  - `removePaymentMethodTest()`: controlla che il metodo di pagamento possa essere rimosso con successo.
- `AdminUserControllerTest`
  - `removeUserTest()`: testa l'eliminazione di un utente dal sistema da parte dell'amministratore.
- `AdminBookControllerTest`
  - `removeBookTest()`: simula la rimozione di un libro da parte dell'amministratore.
- `AdminExtraControllerTest`
  - `resetDatabaseTest()`: verifica il corretto reset del database tramite lo script `reset.sql`.

Tutti i test sono stati progettati per validare sia casi standard che condizioni di errore, assicurando la stabilità e la correttezza della logica applicativa di BookTrack.

```java
class LoginControllerTest {

    @Test
    void loginTest() throws SQLException, ClassNotFoundException {

        LoginController loginController = new LoginController();

        // Cas de test 1 : utilisateur connu
        String username = "nicolas.dupont";
        String password = "SecurePass123";

        try {
            User user = loginController.login(username, password);
            assertNotNull(user, "L'utilisateur devrait être trouvé avec des identifiants valides.");
        } catch (SQLException | ClassNotFoundException e) {
            fail("Exception levée lors de la tentative de connexion : " + e.getMessage());
        }

        // Cas de test 2 : utilisateur inconnu
        String unknownUsername = "fake.user";
        String unknownPassword = "FakePassword";

        try {
            User unknownUser = loginController.login(unknownUsername, unknownPassword);
            assertNull(unknownUser, "Aucun utilisateur ne devrait être trouvé.");
        } catch (SQLException | ClassNotFoundException e) {
            fail("Exception levée pour un utilisateur inconnu : " + e.getMessage());
        }
    }
```

Figura 29: loginTest della classe loginControllerTest

UserBookPageControllerTest

I test della classe `UserBookPageControllerTest` verificano il corretto funzionamento dei metodi `addBook()` e `cancelReservation()`, utilizzando utenti e libri già presenti nel database di default.

Questi test servono a garantire che:

- un utente possa prenotare un libro disponibile senza errori, con verifica anche del metodo di pagamento associato;
- un utente possa annullare una prenotazione già effettuata e che il sistema aggiorni correttamente la disponibilità del libro e lo stato della prenotazione nel database.

I test includono sia casi positivi (operazioni valide) che negativi (prenotazioni duplicate, tentativi di cancellazione su libri non prenotati, ecc.), per assicurare robustezza e affidabilità delle funzionalità di prenotazione.

```
class LoginControllerTest {

    @Test
    void loginTest() throws SQLException, ClassNotFoundException {

        LoginController loginController = new LoginController();

        // Cas de test 1 : utilisateur connu
        String username = "nicolas.dupont";
        String password = "SecurePass123";

        try {
            User user = loginController.login(username, password);
            assertNotNull(user, "L'utilisateur devrait être trouvé avec des identifiants valides.");
        } catch (SQLException | ClassNotFoundException e) {
            fail("Exception levée lors de la tentative de connexion : " + e.getMessage());
        }

        // Cas de test 2 : utilisateur inconnu
        String unknownUsername = "fake.user";
        String unknownPassword = "FakePassword";

        try {
            User unknownUser = loginController.login(unknownUsername, unknownPassword);
            assertNull(unknownUser, "Aucun utilisateur ne devrait être trouvé.");
        } catch (SQLException | ClassNotFoundException e) {
            fail("Exception levée pour un utilisateur inconnu : " + e.getMessage());
        }
    }
```

Figura 30: UserBookPageController con loginTest()

```java
class UserBookPageControllerTest {

    @Test
    void addBookTest() throws SQLException, ClassNotFoundException {

        String username = "mario.rossi";
        String password = "Password1";

        LoginController loginController = new LoginController();
        User user = loginController.login(username, password);

        UserBookPageController userBookPageController = new UserBookPageController(user);

        // Test case 1: valid book addition
        String title = "Test Title";
        String author = "Test Author";
        String publisher = "Test Publisher";
        String publishDate = "2030-01-01";
        boolean isReturnable = true;
        float price = 10;

        ByteArrayOutputStream outContent = new ByteArrayOutputStream();
        PrintStream originalOut = System.out;
        System.setOut(new PrintStream(outContent));

        try {
            userBookPageController.addBook(title, author, publisher, publishDate, isReturnable, price);
        } catch (SQLException e) {
            System.err.println("Error: " + e.getMessage());
        }
```

Figura 31: UserBookPageController con addbooktest()

Anche i metodi delle classi `UserBookManagementControllerTest` e `UserProfileControllerTest` lavorano allo stesso modo, verificando il corretto funzionamento delle operazioni di aggiornamento dei dati personali e dei metodi di pagamento degli utenti registrati nel sistema *BookTrack*.

Ad esempio, il metodo `requestBookCancellationTest()` esegue tre test case principali:

1. **Prenotazione esistente con richiedente come creatore:** verifica che un utente possa effettivamente richiedere l'annullamento di una prenotazione di un libro che ha effettuato personalmente;

2. **Prenotazione esistente ma con richiedente diverso dal creatore:** controlla che un utente non possa annullare una prenotazione effettuata da un altro utente;

3. **Prenotazione inesistente:** gestisce il caso in cui si tenti di annullare una prenotazione non presente nel database, garantendo la corretta risposta del sistema in termini di gestione degli errori.

Questi test assicurano che le funzionalità di gestione dei libri e del profilo utente siano coerenti con i requisiti del sistema e che non siano consentite operazioni improprie o non autorizzate.

```
void requestBookCancellationTest() throws SQLException, ClassNotFoundException {

    String username = "mario.rossi";
    String password = "Password1";

    LoginController loginController = new LoginController();
    User user = loginController.login(username, password);

    UserBookPageController userBookPageController = new UserBookPageController(user);

    // Test case 1: user owns the book
    ByteArrayOutputStream outContent = new ByteArrayOutputStream();
    PrintStream originalOut = System.out;
    System.setOut(new PrintStream(outContent));

    userBookPageController.requestBookCancellation(1);

    System.setOut(originalOut);
    String output = outContent.toString();
    assertTrue(output.contains("Request submitted." + "\nCancellation request saved."));

    // Test case 2: user doesn't own the book
    ByteArrayOutputStream outContent2 = new ByteArrayOutputStream();
    PrintStream originalOut2 = System.out;
    System.setOut(new PrintStream(outContent2));

    userBookPageController.requestBookCancellation(2);

    System.setOut(originalOut2);
```

Figura 32: RequestBookCancellationTest()

I metodi delle classi `AdminUserControllerTest`, `AdminBookControllerTest` e `AdminExtraControllerTest` sono progettati per testare il corretto funzio-

namento delle operazioni amministrative, rispettivamente legate alla gestione degli utenti, dei libri e del database.

L'implementazione dei test segue una logica simile a quella descritta per le altre classi di test:

- I metodi di `AdminUserControllerTest` verificano che un amministratore possa rimuovere correttamente un utente registrato.
- I metodi di `AdminBookControllerTest` assicurano che l'amministratore possa eliminare un libro dal sistema, considerando anche eventuali vincoli come prestiti attivi o prenotazioni associate.
- Infine, i metodi di `AdminExtraControllerTest` testano le funzionalità relative al reset del database e al caricamento dei dati di default, utili per garantire l'integrità e la riusabilità dell'ambiente di test.

Questi test sono fondamentali per assicurare che il sistema *BookTrack* offra un controllo completo e sicuro delle funzionalità di amministrazione.

4.2 CoreEntities Test

Sono stati implementati test unitari per verificare il corretto comportamento delle classi del modello di dominio, in particolare per quanto riguarda la gestione dei pagamenti e dei rimborsi:

- `CreditCardTest`: `payTest()` e `refundTest()`
- `PayPalTest`: `payTest()` e `refundTest()`

Questi test verificano il corretto funzionamento dei metodi `pay()` e `refund()` delle classi `CreditCard` e `PayPal`, attraverso una simulazione realistica: un utente con un metodo di pagamento associato effettua una prenotazione per un libro a pagamento.

Nel dettaglio:

- Nei test `payTest()`, si verifica che:
  - Prima dell'operazione, non esiste alcuna prenotazione associata all'utente.
  - Dopo l'esecuzione del metodo `pay()`, la prenotazione viene correttamente registrata nel sistema.

- Nei test `refundTest()`, si verifica che:
  - L'utente abbia effettivamente una prenotazione attiva prima del rimborso.
  - Dopo l'esecuzione del metodo `refund()`, la prenotazione viene rimossa, simulando un rimborso riuscito.

Inoltre, i metodi `pay()` e `refund()` sono stati anche invocati nei test di integrazione definiti nella classe `UserBookPageControllerTest`, in particolare:

- `attendBookTest()` per testare il pagamento al momento della prenotazione.
- `removeReservationTest()` per simulare il rimborso dopo l'annullamento.

Questi test aiutano a garantire che il sistema di prenotazione e pagamento funzioni correttamente, sia dal punto di vista logico (modello di dominio) che funzionale (integrazione col database).

```
void payTest() {
    User user = new User(1, "Alice", "Bianchi", 25, "alice.bianchi",
            "alice.bianchi@gmail.com", "SecurePass1", "Credit Card",
            "987654321", "2029-06-30", "999");
    Book book = new Book("Clean Code", "Robert C. Martin", "Pearson", "2025-01-01", true, 25.0f, 5);

    String userInput = "yes\n999\n";
    System.setIn(new ByteArrayInputStream(userInput.getBytes()));

    ByteArrayOutputStream outputBuffer = new ByteArrayOutputStream();
    PrintStream originalOut = System.out;
    System.setOut(new PrintStream(outputBuffer));

    user.getPaymentMethod().pay(book);

    System.setOut(originalOut);
    String output = outputBuffer.toString();

    assertTrue(output.contains("Paying for book: Clean Code"));
    assertTrue(output.contains("Amount: 25.25€")); // includes 1% commission
    assertTrue(output.contains("Payment successful!"));
}
```

Figura 33: metodo payTest()

```
void refundTest() {
    User user = new User(1, "Alice", "Bianchi", 25, "alice.bianchi",
            "alice.bianchi@gmail.com", "SecurePass1", "Credit Card",
            "987654321", "2029-06-30", "999");
    Book book = new Book("Clean Code", "Robert C. Martin", "Pearson", "2025-01-01", true, 25.0f, 5);

    ByteArrayOutputStream outputBuffer = new ByteArrayOutputStream();
    PrintStream originalOut = System.out;
    System.setOut(new PrintStream(outputBuffer));

    user.getPaymentMethod().refund(book);

    System.setOut(originalOut);
    String output = outputBuffer.toString();

    assertTrue(output.contains("Refunding payment for book: Clean Code"));
    assertTrue(output.contains("Refund successful!"));
}
```

Figura 34: metodo refundTest()

Elenco delle figure

1 Package Dependency Diagram 2
2 Use Case Diagram 3
3 Prototipo della pagina di Sign in – MK#1 12
4 Prototipo della pagina di Registrazione – MK#2 13
5 Prototipo della pagina di prenotazione dei libri – MK#4 14
6 Prototipo della pagina di pagamento – MK#4 15
7 Prototipo della pagina di pagamento con credenziali personali . 16
8 Prototipo della pagina di gestione dei libri (Admin) 17
9 Diagramma delle classi del package `LibraryManagement` 19
10 Diagramma delle classi del package `CoreEntities` 20
11 Diagramma delle classi del package `ORM` 21
12 ER Diagram iniziale 23
13 Schema ER ristrutturato 23
14 Navigation Diagram del sistema BookTrack 26
15 Struttura delle directory del progetto BookTrack 27
16 Implementazione delle classi Reservation e ReservationMapper 30
17 implementazione class AdminBookController 33
18 implementazione class AdminUserController 34
19 implementazione class AdminExtraController 35
20 implementazione class DatabaseConnectionManager 36
21 Costruttore classe UserDao con metodi addUser,getUserById,updateEmail 38
22 class AdminDAO con metodo updatepassword 39
23 class AdminDAO con metodo resetdtbase genratedatabase . . . 40
24 class BookDAO con metodo addBook removeBook 41
25 class CreditCardDAO 43
26 class PayPalDAO 44
27 ricreazione e rimozione della tabella user nel file reset$_{b}ookstrack_{d}ata.sqlpaypalecreditca$
28 implementazione pagina iniziale e sign in 47
29 loginTest della classe loginControllerTest 50
30 UserBookPageController con loginTest() 51
31 UserBookPageController con addbooktest() 52
32 RequestBookCancellationTest() 53
33 metodo payTest() 55
34 metodo refundTest() 56

Elenco delle tabelle

1 Use Case #1 – Accesso al sistema (Sign in) 4
2 Use Case #2 – Registrazione del sistema (Sign up) 5
3 Use Case #3 – Partecipazione a un libro condiviso 6
4 Use Case #4 – Pagamento della quota di prenotazione 7
5 Use Case #5 – Inserimento di un nuovo libro 8
6 Use Case #6 – Cancellazione di una prenotazione 9
7 Use Case #7 – Aggiornamento del metodo di pagamento 10
8 Use Case #8 – Reset del database 11